

EdgePilot AI

Complete documentation - the project record and every folder in the project

EdgePilot AI - Team 15

Documentation set, 26 August 2026

89 documents, 88 folders

EdgePilot AI - Project Record

Every change made to this codebase, the part of the system it touched, what was wrong before it, and what the structure looks like now against the version the team shipped.

Project: EdgePilot AI - compare local and cloud AI deployment with real measurements. **Team:** 15. **This working copy:** Documents/Air On Accountant Backup/EdgePilot testing/app. **Compared against:** Desktop/Air on Accountants/edgepilot-ai-main (1)/edgepilot-ai-main, the team's repository after their updates. **Record written:** 23 August 2026, at commit b13af6e.

1. How to read this document

Section 3 is the fix list. Section 4 is the structural comparison. Section 5 is the argument for why this structure is the better one, and section 6 is the honest list of what it still does not do.

How a change is placed. Every fix below names the **work package** it lands in, not a person. The purpose is navigational: it tells you which part of the system to open, and therefore which conversation the change belongs to. The areas are the ones the repository already draws - the module a file sits in - so nothing here depends on anyone's recollection. Where a fix crosses two areas, both are named.

Verification method. Every "before" state in section 3 is a real file in the comparison copy on the Desktop, and every "after" state is a real file here. A claim of the form "this returned a constant" can be checked by opening both. Line counts come from diff between the two trees.

2. Where each work package lives

Use this to find the part of the system a change belongs to, and therefore which conversation it needs.

Benchmark and provider layer - measurement, the provider adapters, scoring and evidence.

src/modules/benchmark/, the benchmarks, providers, readiness, comparisons and local-runtime endpoints under src/app/api/v1/, scripts/benchmark/, scripts/evaluation/, docs/benchmark/, docs/local-model-setup.md.

Integration and API surface - routes, shared types, the database schema and the internal documentation.

src/app/api/v1/workloads/, src/shared/types/, prisma/, docs/internal/.

Product UI and benchmark dashboard - the four-step user journey and its styling. src/app/dashboard/, src/components/dashboard/, docs/dashboard/.

Vision benchmark - image classification end to end, including the dataset. src/modules/vision-benchmark/, src/app/vision-benchmark/, src/components/vision/, scripts/vision-benchmark/, docs/vision-benchmark/, datasets/.

Device knowledge and quality - removed in this pass; see change 22. The former src/modules/device/, src/app/api/v1/devices/, device-profile.json.

DevOps - the container, CI, the shared-database guards and the evidence page. Dockerfile, .dockerignore, .github/workflows/, scripts/db/, src/app/evidence/.

These are the divisions the repository itself already uses: AI_USAGE.md is organised one section per work package, and docs/benchmark/README.md and docs/dashboard/README.md are each written as a module guide.

3. The fix list

Forty-one changes, grouped by what they were about. Small ones are included - a one-character regex fix is in here, because it was the difference between a feature working and a feature that had never once worked.

3.1 Measurement correctness

These are the changes that decide whether a number in this application means anything.

1. hardwareFit was a constant. *Area: benchmark layer.* Every device on earth received `hardwareFit: 50`, from a Raspberry Pi to a threadripper. It is visible in the committed evidence: `evidence/benchmark/measured-ollama-llama3.2_latest.json` still records `"hardwareFit": 50`, because that artefact predates the fix. Replaced with `src/modules/benchmark/core/services/HardwareFitAssessor.ts` (229 lines, new), which scores *placement* from what the runtime reports, and `src/modules/benchmark/infrastructure/OllamaResidencyProbe.ts` (107 lines, new), which reads it. Ollama's `/api/ps` gives `size` (total resident bytes) and `size_vram` (how many of those are on the GPU); `size_vram / size` is the fit signal, and it is reported by the runtime rather than inferred by us.

2. Hardware fit deliberately does not score speed. *Area: benchmark layer.* Latency is already measured and already a readiness component. If hardware fit also punished CPU execution, a slow run would be penalised twice and the total would stop meaning anything. Recorded in the file's own header so the next person does not "fix" it back.

3. VRAM taken and spill share are now stated in words. *Area: benchmark layer.* The assessment says how much VRAM the model took and, on a partial offload, how many bytes did not fit and what percentage of the model that was. Where the runtime does not report card capacity, the assessment says so rather than presenting a share of an unknown total.

4. Cold start contaminated every first iteration. *Area: benchmark layer.* A model that is not resident has to be loaded from disk before it can answer. Measured on the same 8B model in the same session: time-to-first-token was **36,445 ms cold** and **369 ms warm** - a factor of 99 - and readiness swung from 75 to 90 on load state alone. Averaging that first iteration into a five-iteration mean does not describe the model; it describes whether someone had used it recently. Fixed in `BenchmarkRunner.ts`: the runner now runs $N+1$ iterations and discards the first. The user is told their chosen iteration count will be raised by one, the discarded cold boot is reported with its own duration and its own row in the results, and the database records it as `warmup: true` rather than deleting it.

5. The success chain counted the discarded iteration. *Area: benchmark layer.* After change 4, a request for three iterations reported "4/4 iterations succeeded". The measured set is now `responses.slice(1)` and `success`, `failure-code` dominance and the summary all read from that. The fix went into the runner rather than the test that caught it.

6. The residency probe result was used without a null guard. *Area: benchmark layer.* The catch produced the correct value for the wrong reason, which would have masked any other fault in the probe. Now before `=== null ? null : before.residentBytes !== null && before.residentBytes > 0`.

7. Privacy was averaged into the readiness score. *Area: benchmark layer.* A coarse label was mapped to 30/60/100 and blended with latency and reliability. That is a category error twice over: it implies a vendor is "twice as private" as another, and it lets a fast provider average away a data policy that should have disqualified it outright. `ReadinessCalculator.ts` dropped from a five-term average to four measured performance terms, and privacy became an ordinal class produced by `PrivacyAssessor.ts` (314 lines, new) that annotates or disqualifies a recommendation instead of nudging a number.

8. A mean was being reported as a result. *Area: benchmark layer.* `ComparisonReport.ts` (506 lines, new) checks every dimension against the per-iteration samples behind it. If the entrants' observed ranges overlap, no winner is declared. At five iterations a 10% gap in means routinely sits inside a 3x spread. A worked consequence: after cold starts were removed, latency did **not** become a cleanly separated dimension - separability *fell* from 0.82 to 0.58, because removing the cold start shrank the gap more than it shrank the spread. The report says so rather than declaring a winner.

9. Two local models were about to be compared in parallel. *Area: benchmark layer.* For two cloud providers, parallel is correct. For two local models it destroys the measurement: both want the same GPU, the runtime may evict one to make room for the other, and the latency that comes back describes contention.

ComparisonPlanner.ts (141 lines, new) decides execution mode before anything runs, and RunComparison.ts honours the plan instead of always calling Promise.all.

10. Nothing stopped a text model being compared to an embedding model. Area: *benchmark layer*.

ModelModality.ts (167 lines, new) classifies from Ollama's families array. Vision (clip, mllama, llava, qwen2vl, vision) and embedding (bert, nomic-bert, gte, jina-bert) are identified positively; text is the residual and is always marked inferred, because "no vision family present" is not proof of anything.

11. task_type was a column nothing read. Area: *benchmark layer, over the integration layer's workload schema*. You could register an image_recognition workload and benchmark an embedding model against it and no part of the system objected. TaskCompatibility.ts (174 lines, new) turns the declaration into a constraint. The rule is deliberately asymmetric: a vision model can do text, a text model cannot do vision, an embedding model can do neither.

12. Timeouts were too low for a cold local model, and for cloud. Area: *benchmark layer*. Raised for the local path so a genuine cold load is not recorded as a timeout, and raised for the online path in the same pass.

13. code_generation no longer implies the code works. Area: *benchmark layer*.

shapeOf('code_generation') returns text that says so: nothing in this system compiles or runs the output, so "it produced code" is not "the code works".

3.2 Identity, ownership and privacy

14. Users were being asked to paste UUIDs by hand. Area: *dashboard, and integration*. The setup step had a field for a workload UUID because the endpoint that should have created one did not persist. Replaced by src/components/dashboard/session.ts (92 lines, new): the browser generates a durable id, stores it in localStorage, sends it as x-edgepilot-session on every request, and **never renders it**. A user should no more be asked to see their own row-ownership key than to type their own cookie.

15. POST /api/v1/workloads never called the database. Area: *integration*. The committed version returned the literal string temp-workload-id, so nothing the dashboard saved reached Postgres and every benchmark run afterwards failed with 404 Workload not found. The route is now 126 lines against the original 53 and writes real rows. The original is kept beside it as route.ts.scaffold.bak and is referenced by name in the new file's header.

16. Every row was attributed to one shared user. Area: *integration*. Two routes attributed everything to local-sandbox@edgepilot.invalid. Harmless on one laptop; a data leak the moment it is hosted, because GET would hand every visitor every other visitor's workloads. src/lib/sessionOwner.ts (76 lines, new) upserts a User per session id. It fails closed: a request without the header gets 400, never a fallback to a shared user.

17. Session activity had nowhere to be recorded, and no redaction rule. Area: *benchmark layer*.

src/core/logging/SessionLog.ts (407 lines, new) records what the application did, exportable as JSON, with no account attached. Prompt content is never recorded by default - only a length and a digest. A log that wrote out the same prompt the egress warning had just flagged would have defeated its own warning. Logging is opt-in by header: a request with no session id is not logged anonymously, it is not logged at all.

18. The privacy report arrived after the prompt had already left. Area: *benchmark layer*. egress-warning.ts (133 lines, new) answers the question before the run: is this prompt about to leave the machine, and to whom. It fails closed on a null endpoint.

19. The egress warning object contained the prompt text. Area: *benchmark layer*. Found by a test written in the same session. An object whose job is to warn that text is about to leave the machine must not carry a copy of that text. It now reports the character count and nothing else. An older assertion in privacy-assessor.test.ts had pinned the opposite behaviour and was reversed, with the reasoning recorded at the call site.

20. The privacy catalogue ships empty on purpose. Area: *benchmark layer*. Every value in privacy-catalogue.ts starts null and unverified. These are policy facts from terms of service, they differ between free and paid tiers, and they change without notice. Filling them in from memory is how an evidence-based tool starts publishing stale claims confidently. Filling them in is a human task and is listed in section 6.

21. The login page was removed. *Area: integration.* There are no accounts, so an auth surface was a door onto an empty room. `src/app/auth/` and `src/app/api/auth/` are gone.

3.3 Removing the device module

22. The device profile was a client-side self-report that nothing could check. *Area: device.* The user typed how much RAM they had; the application stored it; no measurement ever used it. Once hardware fit became `size_vram / size` read from the runtime, the typed value had no consumer and could only disagree with reality. Removed to the roots rather than left nullable: the UI panel, the API route, the module, the types, the tests, and the database.

23. The database migration. *Area: device; the migration came out of the benchmark and DevOps work.* `prisma/migrations/20260823035500_drop_devices/` drops two foreign keys, the `benchmarks_device_id_idx` index, the `benchmarks.device_id` column and the `devices` table. Tested on the local sandbox database before it was applied to the shared Neon instance, and applied there with `migrate deploy - never migrate dev`, which can offer to reset a shared database.

24. Two dev scripts still referenced devices, and next build caught them. *Area: benchmark layer.* `scripts/dev/show-last-run.ts` read `run.deviceId` and `scripts/dev/seed-demo-rows.ts` called `prisma.device.create`. Jest never touched either file. This is the reason `npm run build` is in the verification list and not just `npm test`.

3.4 The vision benchmark

25. The live evidence path had never worked, once, ever. *Area: vision.* `evidence-store.ts` validated filenames against `/^[a-z0-9._-]+\..json$/` - lowercase only - and then built the filename from an ISO timestamp containing an uppercase T and Z. Every live run failed to write its evidence. One `.toLowerCase()` fixed it. It is the smallest change in this document and it is the reason any live measurement exists at all.

26. The first vision sample paid the cold-start cost. *Area: vision; the pattern came from the benchmark runner.* The executor now runs a warm-up classification before the measured set. Median latency across the dataset fell from 1322.1 ms to 529.6 ms between the two live runs - the same model, the same images, the difference being that sample one was no longer carrying the model load.

27. The warm-up was wrong three times before it was right. Recorded because the mistakes are instructive. It consumed a deterministic test clock (fixed by calling `provider.classify()` directly rather than the timed wrapper); it fired on cloud providers, which meant a billable call and one extra image sent to a third party (fixed: local providers only, plus an explicit `warmUp?: boolean`); and it ate scripted responses in two existing tests (fixed by opting those two out with a comment, and adding two new tests that cover the behaviour properly).

28. Users can now bring their own dataset, and it never leaves their machine. *Area: vision.* `src/components/vision/DatasetUpload.tsx` (403 lines, new) takes a folder-per-class upload. `browser-image-processor.ts` (181 lines, new) resizes and hashes in the tab; `browser-ollama-provider.ts` (171 lines, new) talks from the tab straight to the user's own Ollama on localhost. No uploaded image ever reaches the EdgePilot server. That is not a convenience decision - benchmarking confidential data is this project's stated non-goal, and shipping images to a server to be resized would have contradicted it.

29. The browser processor's version string is deliberately different. *Area: vision.* It is `browser-canvas-png-v1`, not `sharp-png-v1`. Canvas and sharp do not resample or encode PNG identically, so two runs prepared differently must not be comparable by accident. Evidence records the preprocessing version for exactly this reason.

30. Arbitrary labels and licences. *Area: vision.* The metrics schema pinned `perClass` to `.length(VISION_LABELS.length)` - seven. Any dataset with a different number of classes ran to completion and *then* failed validation. Six fields were widened in one sweep, TypeScript interfaces and zod schemas together, after three separate build round-trips caused by widening one without the other. All three pre-existing evidence files were re-parsed afterwards to confirm backward compatibility.

31. The prompt has to come from the dataset. *Area: vision.* A deliberate failure run proved it: with the built-in prompt and a custom two-class dataset the result was 0% accuracy and 100% invalid output.

`buildVisionPrompt(labels)` builds the closed label set from the dataset. Re-run with `red_panel/green_panel: 100%`, passed.

32. A loose file at the chosen root took the root folder's name as its class. *Area: vision.* `labelOf` now requires three path parts, so only `root/class/file` is accepted.

33. Four declarations block the run. *Area: vision.* SPDX licence text plus three privacy checkboxes. A dataset with no stated licence and no confirmation of what is in the images is not a dataset anyone should publish a metric from.

34. The real llava measurement, recorded as a failure. *Area: vision.* 21 samples, 14 correct: **66.7% accuracy, macro-F1 0.667**, 0% invalid output, 100% request success, median 529.6 ms, p95 721.4 ms. It **fails** its own gates (0.80 accuracy, 0.75 macro-F1) and the artefact records `"passed": false`. The per-class matrix closes exactly: `hardhat` has three false negatives against `safety_cone`'s three false positives - all three `hardhats` were called `safety_cones`, and `hardhat` scores F1 0.0. Two gloves and two masks were called `hardhats`. This is kept in the repository deliberately; see change 40.

3.5 Build, evidence and repository hygiene

35. .next held 142 stale files pointing at an old Downloads path against 75 correct ones, which is what the React Client Manifest errors were. Cleared.

36. .dockerignore was excluding something the build needed. *Area: DevOps.* Fixed and proven: `docker build -t edgepilot-ai:test .` completes in 154.8 s with `RUN npm run build` succeeding inside the image.

37. The evidence page traced the whole project into the standalone bundle. *Area: DevOps.* A dynamic `path.join(process.cwd(), root)` inside a loop defeats Next's file tracing. Rewritten with literal path segments.

38. /evidence exists as a page. *Area: DevOps.* `src/app/evidence/page.tsx` (286 lines, new) leads every artefact with `what_this_proves` and `what_this_does_not_prove` before showing a single number.

39. The ten-case evaluation matrix was missing. *Area: benchmark layer.* `scripts/evaluation/ten-case-matrix.ts` (340 lines, new), run with `npm run eval:matrix`, drives real module code - the same zod schemas, normaliser and readiness calculator the application uses - across 10 cases in 6 categories (1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure) and exits non-zero on failure. Current artefact: all 10 behaved as documented, no failures. Case 9 exists to assert that a missing hardware fit stays `null` and is never rendered as `0`.

40. Working backups and the one real measurement were both mishandled by git. *Area: DevOps.* Nineteen `.bak` files sat beside real source, and ten of them were already tracked - so a `.gitignore` rule alone would not have removed them. Eighteen were cleared out and `*.bak` was added to `.gitignore`. In the same rule set, `/evidence/vision-benchmark/live-*.json` was silently ignoring the llava run - the first real measurement the project produced. Both decisions are now encoded as negation rules with the reasoning written beside them, rather than as `git add -f` flags someone has to remember.

41. scripts/db/ guards the shared database. *Area: DevOps.* `dotenv -e .env.neon -- prisma migrate deploy` fails *silently* when the file is absent: `dotenv` loads nothing, `Prisma` falls back to `.env`, and a command named "neon" runs against localhost while reporting success. `require-env-file.mjs` refuses to proceed unless the file exists, parses, and names a host that is not local; `neon.mjs` reads the file itself and prints the destination host before doing anything.

4. This structure against the team's

Compared with `Desktop/Air on Accountants/edgepilot-ai-main (1)/edgepilot-ai-main`, excluding `node_modules`, `.next`, `.git`, `package-lock.json` and `.env`. Fifty-five files differ, fifty-two paths exist only here - two of them local scratch (`.env.neon`, `_to_delete/`) rather than project content - and twenty-six exist only there.

4.1 What is gone

Removed	Why
src/modules/device/, src/app/api/v1/devices/, src/shared/types/device.ts, tests/device/	Self-reported hardware that no measurement consumed - see change 22
src/app/auth/, src/app/api/auth/	There are no accounts; ownership is a browser-generated session id
sentry.client.config.ts, sentry.edge.config.ts, sentry.server.config.ts	Three config files for a service with no DSN and no project behind it
src/core/rate-limit/, src/core/security/	A rate_limits table and a middleware layer guarding endpoints that have no authentication and no public URL
src/data/	A parallel data-access path beside src/modules/*/infrastructure/repositories/
src/components/ErrorBoundary.tsx, Loading.tsx, LoadingStates.tsx, providers/	Three overlapping loading components; the App Router's own loading.tsx and error.tsx conventions cover it, and StateViews.tsx holds the rest
src/core/logging/logger.ts	Superseded by SessionLog.ts, which has a redaction rule and an export format
AUDIT_REPORT.md, FINAL_AUDIT_REPORT.md, PRODUCTION_AUDIT_REPORT.md, REMAINING_PRODUCTION_GAPS.md, TEST_DOCUMENTATION.md	Five root-level audit documents describing the same codebase at five moments, none marked superseded. Replaced by this one record plus per-folder documentation
public/index.html, public/favicon.svg	A static page shadowing an App Router route
docs/deployment.md	Described a deployment that does not exist yet; the honest version is section 6

4.2 What is new

Added	What it does
src/modules/benchmark/core/services/ - 8 new files, 1,761 lines	Hardware fit, modality, task compatibility, privacy class, egress warning, policy catalogue, comparison planner, comparison report
src/modules/benchmark/infrastructure/OllamaCatalog.ts, OllamaResidencyProbe.ts	Runtime status, installed models, and the memory reading that hardware fit is computed from
src/app/api/v1/comparisons/, local-runtime/, session-log/, session-log/share/	Four endpoints the original did not have
src/app/evidence/page.tsx	Artefacts readable without running anything
src/core/logging/SessionLog.ts, sessionLogStore.ts	Redacting, opt-in, exportable session record
src/lib/sessionOwner.ts, src/components/dashboard/session.ts	Account-free row ownership
src/components/dashboard/InstalledModels.tsx, VisionHandoff.tsx	Always-visible model panel with per-step filtering; handoff into the vision dashboard
src/components/vision/DatasetUpload.tsx and the two browser vision adapters	Bring-your-own dataset that never leaves the machine
scripts/db/, scripts/dev/, scripts/evaluation/	Shared-database guards, local helpers, the ten-case matrix
tests/core/, tests/evaluation/, plus new benchmark and dashboard suites	322 tests across 29 suites, from 259
4 Prisma migrations	Nullable hardware fit and shared findings; optional email and session owner; drop devices; warm-up iteration

Added	What it does
AI_USAGE.md	Required submission item, one section per work package

4.3 What changed in place

The twenty largest, by lines differing:

File	Theirs	Ours	Lines differing	Work package
src/app/globals.css	418	27	431	Dashboard
src/app/dashboard/page.tsx	692	15	697	Dashboard
src/components/dashboard/SetupPanel.tsx	322	201	379	Dashboard
src/app/vision-benchmark/page.tsx	73	244	293	Vision
src/modules/benchmark/application/services/BenchmarkRunner.ts	469	662	269	Benchmark
prisma/seed.ts	127	56	137	Integration
src/app/api/v1/workloads/route.ts	53	125	126	Integration
src/app/page.tsx	97	36	123	Dashboard
src/components/dashboard/RunPanel.tsx	214	315	119	Dashboard
src/components/dashboard/ProviderPanel.tsx	219	294	113	Dashboard
src/modules/benchmark/core/services/ReadinessCalculator.ts	40	88	110	Benchmark
src/components/dashboard/DashboardApp.tsx	134	203	91	Dashboard
src/modules/benchmark/application/use-cases/RunBenchmark.ts	316	308	90	Benchmark
tests/vision-benchmark/vision-execution.test.ts	301	361	86	Vision
prisma/schema.prisma	167	199	82	Integration
src/components/dashboard/api.ts	202	242	78	Dashboard
tests/benchmark/run-benchmark.test.ts	488	483	73	Benchmark
src/components/dashboard/RunResults.tsx	358	413	69	Dashboard
src/modules/vision-benchmark/application/executor.ts	159	222	65	Vision
src/modules/vision-benchmark/core/types.ts	159	201	62	Vision

src/app/dashboard/page.tsx going from 692 lines to 15 is not deletion. The route file is now a route file - metadata and a mount point - and the 692 lines of behaviour live in src/components/dashboard/, where they can be tested without a request.

5. Why this structure is the better one

5.1 The boundaries are real, not decorative

Both versions describe themselves as hexagonal. In this one the description holds when you check it. src/modules/benchmark/core/ imports nothing from infrastructure/; the ports are two interface files, AIPProvider.ts and BenchmarkRepository.ts; and infrastructure/container.ts is the single place where an adapter is constructed. No API route in this tree builds a provider, reads an environment variable, or touches Prisma. That is what makes the provider layer testable without a network: tests/benchmark/helpers.ts injects a fake fetch and a stepped clock, and the suite is deterministic because nothing underneath it can reach out.

The version with src/data/ beside src/modules/*/infrastructure/repositories/ had two answers to "where does data access live", which is the same as having none.

5.2 Every number carries its provenance

measured, derived, declared, unverified are values in the type system, not comments. A reader can tell a stopwatch reading from an arithmetic derivation from a placeholder without leaving the JSON. The rule that follows from it is the one that costs the most to keep: **null is never coerced to 0**. Reporting 0 ms for "we never got an answer" is the single most misleading number this system could produce, and there is a test file (tests/dashboard/format.test.ts) whose entire job is to stop it happening in the UI, and a case in the ten-case matrix that asserts it in the data.

A structure where hardwareFit was the constant 50 could not have supported that rule, because the field looked like a measurement and was not one.

5.3 It deletes

The clearest structural difference is that one tree accumulates and the other removes. Five audit reports at the root, three Sentry configs with no DSN, a rate limiter in front of unauthenticated endpoints, three loading components - each of those was reasonable when it was written, and none of them was ever taken back out. Dead structure is not free: it is read by reviewers as current, it appears in searches, and it makes the live parts harder to find.

Removing the device module cost a migration, a UI change, a module deletion and a test deletion. Leaving the column nullable would have cost nothing today and misled every reader afterwards.

5.4 The tests aim at refusals

The interesting question for a measurement tool is not "does it work" but "what does it refuse to say". So the suite is written against refusals: refuse to call a provider for a workload that does not exist; refuse to attribute a run to the wrong user; refuse to discard a measurement because the database went down mid-run; refuse to declare a comparison winner inside the noise; refuse to report a mean when the only iteration was a cold start.

That framing is also why `npm run build` is a required check alongside `npm test`. Jest type-checks only what it imports. `next build` type-checks the whole project, and it is what caught the two dev scripts in change 24 and the six schema-versus-interface mismatches in change 30 - nine defects that a green test suite could not see.

5.5 Evidence is an artefact, not a claim

evidence/ holds files a reviewer can read without installing anything, and each leads with what it does **not** prove. The vision run that fails its own gate is committed *because* it fails: a benchmarking tool whose only published measurement is a pass is not demonstrating a benchmark, it is demonstrating selection.

5.6 The client/server line is load-bearing

infrastructure/config.ts refuses to evaluate in a browser bundle, because every value it holds is a secret or a host. The two browser vision adapters exist for the inverse reason: images must *not* travel to a server, so the processor and the provider run in the tab. Both directions come from the same rule - decide where a thing is allowed to be, then make the wrong location fail loudly.

6. What this still does not do

Listed so nobody has to discover it in a review.

- **No cloud provider path has ever executed.** One Gemini or Groq key would light up the egress warning, tier-aware privacy and free-versus-paid comparison in a single run.
- **The privacy catalogue is still null and unverified.** Deliberate - see change 20 - and a human task.
- **official_source never reaches a recommendation**, which leaves an acceptance criterion open.
- **quality_score exists nowhere**, and is a required structured field.
- **No public URL.** The Docker image builds; nothing is hosted.
- **~~No UI for comparisons, session-log or session-log/share.~~ Closed.** /compare drives the comparison engine, /history drives the session log and its consent-gated share, and /evaluation renders the ten-case matrix. All three endpoints had working backends and no way in.

- **No adversarial or injection tests in the Jest suite.** The ten-case matrix covers the artefact side; the unit suite does not.
 - **Two deliberate conflicts with the specification**, recorded rather than hidden: the device profile registry was a mandatory MVP item and was removed, and `readiness_score()` was specified to weigh privacy, which it no longer does. Sections 3.1 and 3.3 are the defence of both.
 - **docs/internal/database.md is stale** - line 147 still says users own devices, and it omits `shared_findings`, `session_id`, `warmup` and `privacy_class`.
 - **device-profile.json is still at the root**, referenced only by `.dockerignore` and two historical evidence files.
 - **describeEgressWarning is tested in two files.** Flagged and left alone deliberately.
-

6a. What changed after this record was written

The record above describes the state at commit `b13af6e`. Three further passes landed on top of it, and the parts that change what section 6 claims are here rather than rewritten into it - a record that quietly edits itself is not one.

The interface arrived. A UI/UX pass added `/compare`, `/evaluation`, theming and an arcade navigation, and closed the "endpoint works, nothing calls it" gap for three endpoints at once. Its only reach into measurement code was `OllamaCatalog`, which gained a per-model resident flag read from `/api/ps` - null when the call fails, never conflated with "not loaded".

The vision workload became runnable from the page. `/vision-benchmark` had a server path and a browser path for your own images, and no button for the dataset the project ships with. It is a server action, off by default behind `VISION_BENCHMARK_IN_APP`, and it writes nothing server-side - the evidence is returned for the browser to keep, so `evidence/vision-benchmark/` still means one thing: the reference measurements committed with the project.

Vision runs gained the metrics the text benchmark had. Hardware fit, read by the same residency probe and scored by the same assessor; and the timings Ollama already returned on every reply and the provider was discarding - prompt-eval time, token counts, tokens per second, model load. Deliberately **no TTFT**: that requires streaming, the vision provider sends `stream: false`, and there is no first-token moment to observe. `promptEvalMs` is the closest honest quantity and does not borrow the name.

Every run is kept. `/history` shows text and code runs, comparisons and vision runs together, in the browser rather than on the server. Before it, the dashboard dropped a completed run on navigation and a comparison - the most expensive operation here - survived nowhere at all.

One correctness bug found and fixed. The comparison accepted the same model twice. Entrants are labelled `provider + model` and the report's tally is keyed by that label, so duplicates collided onto one key: it would have declared a winner between a model and itself, decided by whichever run happened to be faster. Refused in `planComparison`, mirrored client-side, pinned by two tests - one that it refuses duplicates, one that `llama3.2:1b` against `llama3.2:3b` still runs, because the rule is about identity and not similarity.

Licensing changed. This work is AGPL-3.0-or-later; the imported baseline keeps its MIT notice, which is a licence obligation rather than a statement about who wrote what.

Still open from section 6: no cloud provider path has executed, the privacy catalogue is still unverified, `official_source` and `quality_score` remain, there is no public URL, and `docs/internal/database.md` is still stale.

6b. Hosted mode

A fourth pass, made for one reason: the site is going to live on Vercel with a Neon database, and most of the Ollama plumbing had been written for a machine where the page and the runtime were the same computer. Section 6a's "no public URL" was about to close, and closing it would have broken every local run for every visitor.

The problem, stated plainly. Every Ollama call except the upload-your-own- dataset path ran on the server and read `OLLAMA_HOST`, which defaults to `localhost`. On a laptop that is the same Ollama the browser sees. Hosted, "localhost" is a container in a datacentre with nothing listening, and a server cannot reach a visitor's laptop. The dashboard, the comparison and the built-in vision run would all have reported "not reachable" for everyone.

Ollama moved into the browser. `infrastructure/browser-ollama.ts` reuses `OllamaCatalog`, `OllamaProvider` and `OllamaResidencyProbe` unchanged - they were plain fetch with no server dependency - and runs them from the tab against the visitor's own machine. The tab does only what it alone can do (the HTTP calls) and sends the raw measurements to the server in a recorded field on the same benchmark and comparison requests. There a `RecordedProvider` replays them through the unchanged `BenchmarkRunner`: same cold-start isolation, same summary, same hardware fit from the residency readings the browser took, same readiness score, same database row, same activity log. A run measured in the browser and one measured on the server are the same kind of evidence, which is what keeps them comparable. No fallback for a recorded run: a cloud model is not a substitute measurement of the visitor's machine. The built-in vision run followed the same path - two new routes serve the dataset description and its 21 images, and the shared executor runs in the browser against the visitor's Ollama.

What cannot be made invisible. Ollama refuses any website it was not told to trust, which is the right default and not ours to override. A visitor allows the site's origin once (`OLLAMA_ORIGINS`), per operating system. `/setup` gives that the room it needs - OS tabs, the exact command with the real origin filled in, a copy button, a connection check - and every place a visitor can hit the refusal (the installed-models panel, the vision page, the home page) points there in one line.

Cloud providers gained real model lists. The provider step used to claim a cloud vendor "does not publish its catalogue" and offered free text with a hardcoded suggestion list that had already drifted from `.env.example`. Both vendors publish it under the same key; `CloudCatalog` asks, narrows to text generators, and marks which accept an image. The dashboard, the comparison and the vision page all pick from real names now, and the placeholder option can no longer be submitted.

Visitors can bring their own keys. Stored in their browser, sent as two request headers, laid over the server's configuration for that one request through a registry that is never cached, never logged, never stored, never echoed. A visitor without keys still uses the site's, and those runs are capped per visitor per hour (core/quota) so the shared quota cannot be drained by one person clicking all evening.

Vision on Gemini and Groq. A Groq vision adapter (OpenAI chat dialect, image as a data URL) joined the Gemini one; the server action takes the provider, the model and the visitor's keys as arguments, because a server action carries no request headers; cloud runs classify seven images at a time so 21 fit a serverless time limit, while a local run stays strictly sequential - two requests to one GPU measure contention, not the model. Committed reference rows stay as they were: controlled Gemini, controlled Ollama, live llava.

Removed. `/api/v1/health/database` and the panel on `/history` that called it. Hosted, an unauthenticated endpoint announcing row counts and migration names to whoever asks is not a diagnostic, and the panel was a debugging aid on a user-facing page. Both are kept outside the repository for local use.

Smaller things that would have broken the deployment. The evidence commit hash falls back to `VERCEL_GIT_COMMIT_SHA` (no `.git` on Vercel); `next.config.mjs` traces `datasets/` into the build (a `readFile` path is not an import, and without this every hosted built-in run fails with `ENOENT`); the vision page sets `maxDuration`; `.env.example` ends with the list a deployment needs, including `BENCHMARK_FALLBACK_ORDER=groq,gemini` so a failed cloud run does not fall back to an Ollama that is not there.

Section 6 after this pass: the cloud provider path now executes from the page (model lists confirmed against live keys; a Groq vision run awaits a key with access to an image-capable model); the privacy catalogue is still unverified; `official_source` and `quality_score` remain; `docs/internal/database.md` is still stale. The public URL is the next step, and the code is now written for it.

7. Verification state at the time of writing

Check	Command	Result
Lint	<code>npm run lint</code>	0 problems
Tests	<code>npm test</code>	348 passed, 33 suites (after 6b)
Build	<code>npm run build</code>	clean, 14 routes
Container	<code>docker build -t edgepilot-ai:test .</code>	succeeded, 154.8 s
Evaluation matrix	<code>npm run eval:matrix</code>	10/10 cases behaved as documented
Vision, live	<code>vision:run:ollama --model=llava:latest</code>	ran, 66.7%, failed its gate , recorded

Two credentials for the shared Neon database were exposed during the work that produced this record and should be rotated. `.env` and `.env.neon` are ignored by `.gitignore` line 28 (`.env*`); `git status --short` must show nothing matching `.env` before any commit.

8. Where the rest of the documentation is

Every folder in this project carries a `FOLDER.md` describing what it is, what happened in it, what each file inside it does, and which other folders it is connected to. Those cross-references are written in both directions, so a trail can be followed either way.

Start at `src/modules/benchmark/FOLDER.md` for the measurement core, or `src/app/FOLDER.md` for the request path.

Eighty-eight folder documents, with cross-folder links, every one of them resolving in both directions - if folder A points at folder B, B carries a "Referenced from" entry pointing back at A.

A PDF set lives in `Documents/AoC Documents`: this record on its own, twelve volumes grouped by area, and one complete 112-page bundle containing everything.

/ - project root

Project: EdgePilot AI, Team 15. **Work package:** shared; each config file belongs to the work package whose tool it configures.

What this folder is

The root holds three kinds of thing and nothing else: the documents that explain the project to a human, the configuration that tells a tool how to build it, and the two data files the application reads at startup. All source lives under `src/`, all tests under `tests/`, all generated proof under `evidence/`.

What happened here

`00-PROJECT-RECORD.md` was added as the single history of the project. It replaces the five separate audit documents the team's version carried at this level (`AUDIT_REPORT.md`, `FINAL_AUDIT_REPORT.md`, `PRODUCTION_AUDIT_REPORT.md`, `REMAINING_PRODUCTION_GAPS.md`, `TEST_DOCUMENTATION.md`), none of which was marked superseded by the others.

Three Sentry configuration files were removed - they configured a service with no DSN behind it. `.dockerignore` was fixed, which is what made `docker build` succeed; `.gitignore` gained a `*.bak` rule and two documented negation rules; `next.config.mjs` grew from 13 lines to 35 to carry the standalone output setting the Dockerfile depends on.

Files

File	What it is
<code>00-PROJECT-RECORD.md</code>	Every fix, the work package it landed in, the structural comparison, and the known gaps. Read this first
<code>README.md</code>	The public entry point - what EdgePilot is and how a user installs it
<code>CONTRIBUTING.md</code>	External contribution guide
<code>AI_USAGE.md</code>	Required submission item: how AI assistance was used per work package, and how the result was verified. One section per work package
<code>LICENSE</code>	Licensing for the whole repository: this work under AGPL-3.0-or-later, and the imported baseline's MIT notice retained as that licence requires
<code>LICENSES/</code>	The full texts - <code>AGPL-3.0.txt</code> and <code>MIT.txt</code>
<code>package.json</code>	Dependencies and the 31 npm scripts. <code>prebuild</code> and <code>pretest</code> regenerate the vision fixtures, so a build and a test run both start from a known dataset
<code>package-lock.json</code>	Exact dependency tree. Committed so CI and a laptop install the same bytes
<code>tsconfig.json</code>	TypeScript configuration. Note <code>target: es5</code> - it is why spreading a <code>Map</code> , <code>Set</code> or typed array is unsafe in this codebase
<code>next.config.mjs</code>	Next.js configuration, including output: <code>'standalone'</code> for the container image
<code>eslint.config.mjs</code>	Lint rules
<code>jest.config.cjs</code>	Test runner configuration
<code>postcss.config.mjs</code> , <code>tailwind.config.ts</code>	Styling pipeline
<code>Dockerfile</code>	Multi-stage container build. Proven working: 154.8 s, with <code>npm run build</code> succeeding inside the image
<code>docker-compose.yml</code>	Local Postgres plus the app
<code>.dockerignore</code>	What the build context excludes. Was the cause of the Docker build failure and is now correct

File	What it is
<code>.gitignore</code>	Includes <code>*.bak</code> and two documented exceptions - the reference llava measurement and the workloads scaffold
<code>.gitattributes</code>	Line-ending normalisation
<code>.env.example</code>	Every environment variable with a comment. The real <code>.env</code> is never committed
<code>benchmark-tasks.json</code>	The controlled task prompts the dashboard offers
<code>device-profile.json</code>	Left over from the removed device module. Referenced only by <code>.dockerignore</code> and two historical evidence files
<code>seed-local-rows.sql</code> , <code>setup-db.sh</code>	Local database bootstrap

Connected folders

- `src/` - everything next `build` compiles.
- `prisma/` - the schema `package.json`'s `db:*` scripts drive.
- `docs/` - the long-form documentation this file summarises.
- `evidence/` - what the `bench:*`, `vision:*` and `eval:matrix` scripts write.
- `scripts/` - the implementations behind those npm scripts.
- `.github/workflows/` - runs `lint`, `test` and `build` from this `package.json` on every push.

Referenced from

These folders point here. Each link below resolves in both directions.

- `.github/`
- `LICENSES/`
- `docs/pdf/`
- `scripts/db/`

`.github/`

`.github/` - repository automation

Work package: integration and API surface, with each check defined by the work package whose script it runs.

What this folder is

GitHub's own configuration directory. It holds one thing: the CI workflow.

What happened here

Unchanged from the team's version. It is listed and documented because the checks it runs are the ones section 5.4 of `00-PROJECT-RECORD.md` argues are load-bearing - in particular `npm run build`, which type-checks the whole project where Jest type-checks only what it imports.

Files

Subfolder only. See `workflows/`.

Connected folders

- `workflows/` - the CI definition.
- `/` - the package `.json` whose scripts CI runs.

LICENSES/ - the full licence texts

Work package: shared.

What this folder is

The verbatim text of both licences this repository is under. LICENSE at the root states which covers what; these are the documents it refers to.

What happened here

The project moved from MIT to **AGPL-3.0-or-later** for the work added after the imported baseline. Two texts are needed rather than one, because two bodies of work are present and neither licence can speak for the other.

The AGPL text was fetched from the Free Software Foundation rather than typed out. A licence is only enforceable if its wording is exact, so reproducing one from memory is not a shortcut - it is a defect with legal consequences.

Files

File	What it is
AGPL - 3.0.txt	661 lines, verbatim from gnu.org. Covers the work added after the baseline commit
MIT.txt	The original notice, preserved unchanged. MIT's single condition is that it travels with copies of the software, so it stays for as long as any baseline file remains

Connected folders

- / - LICENSE states which licence covers which commit, and package.json declares AGPL-3.0-or-later.

datasets/ - benchmark input data

Work package: vision benchmark.

What this folder is

The controlled data a benchmark is run against. One dataset lives here today, for the vision workload.

What happened here

Not restructured. What changed is what the code around it will now accept: the vision module used to require exactly this dataset's seven labels, and a dataset with a different number of classes would run to completion and then fail validation. That restriction was removed, so this folder is now the project's *reference* dataset rather than its only possible one - a user can supply their own from the browser, and it never touches the server.

Files

Subfolder only. See `vision-benchmark/`.

Connected folders

- `datasets/vision-benchmark/` - the dataset itself.
- `src/modules/vision-benchmark/` - the code that loads and measures against it.
- `scripts/vision-benchmark/` - the generator that produces the images and the validator that checks them.
- `docs/vision-benchmark/` - the dataset card and workload specification.

docs/ - long-form documentation

Work package: shared; each subfolder belongs to the module it describes.

What this folder is

The written record: architecture, wire contracts, setup, research reasoning, and per-module guides. 00-PROJECT-RECORD.md at the root is the history; this folder is the reference.

What happened here

docs/deployment.md was removed. It described a deployment that does not exist - there is no public URL - and a document that describes a thing into existence is worse than an admission that the thing is missing. The honest version is section 6 of 00-PROJECT-RECORD.md.

docs/internal/README.md was updated to point at the current set.

One document in here is **known stale** and is listed as a gap rather than quietly patched:

docs/internal/database.md line 147 still says users own "devices", and it does not mention shared_findings, session_id, warmup or privacy_class.

Files

Subfolder / file	What it is
local-model-setup.md	352 lines. Clean machine to measured benchmark: Docker, WSL2, Ollama, API keys, verification, troubleshooting, and the error-code table
benchmark/	The benchmark and provider layer guide, plus its research log
dashboard/	The dashboard module guide
internal/	Architecture, database, wire contract, team workflow, project plan
vision-benchmark/	Dataset card, workload spec, provider execution, test cases, evidence report

Connected folders

- / - README.md and CONTRIBUTING.md are the public-facing counterparts to this folder.
- src/modules/benchmark/ and src/modules/vision-benchmark/ - the code these documents describe.

Referenced from

These folders point here. Each link below resolves in both directions.

- docs/pdf/
- tests/benchmark/

evidence/ - generated proof

Work package: benchmark and DevOps, with the vision artefacts belonging to the vision benchmark.

What this folder is

Artefacts a reviewer can read **without running anything**. That is the whole design constraint. A passing test proves a behaviour to whoever runs the suite; a committed artefact proves it to whoever opens the pull request.

Every file here is written by a script, never by hand, and every one of them leads with two fields: `what_this_proves` and `what_this_does_not_prove`. The second field is the reason the folder is worth trusting.

What happened here

`evidence/evaluation/` is new - it holds the ten-case matrix the submission checklist asks for, which previously existed only as behaviour scattered across the unit suite.

Two live vision measurements were produced, the first ones the project has ever had. They exist only because of a one-character fix: the evidence writer validated filenames as lowercase-only and then built them from an ISO timestamp containing T and Z, so the live path had never once succeeded.

The `.gitignore` rule `/evidence/vision-benchmark/live-*.json` was silently excluding those runs. One is now explicitly un-ignored with the reasoning written beside the rule.

Files

Subfolders only.

Subfolder	What is in it
<code>benchmark/</code>	Measured provider runs, failure-mode captures, the clean-start log
<code>evaluation/</code>	The ten-case evaluation matrix
<code>vision-benchmark/</code>	Controlled and live image-classification runs

Connected folders

- `scripts/` - everything here is written by something there.
- `src/app/evidence/` - the page that renders these files in the browser.

Referenced from

These folders point here. Each link below resolves in both directions.

- `/`

prisma/ - database schema, migrations and seed

Work package: integration and API surface. The four newest migrations came out of the benchmark and DevOps work.

What this folder is

The single definition of the database. Table and column names are snake_case in Postgres and camelCase in TypeScript; the @map() directives in the schema do the translation, which is why nothing in the repository layer has to.

That mapping is contractual, not cosmetic. providers.is_active is what /api/v1/providers returns as is_active, so renaming a @map() value without a migration breaks the wire contract as well as the database.

What happened here

Four migrations were added, and one of them removes a table rather than adding one.

The schema grew by 32 lines net (167 to 199), but the shape of the change matters more than the size: benchmarks.device_id and the whole devices table are gone, users.session_id arrived so rows can be owned without accounts, benchmark_results.warmup arrived so a discarded cold-start iteration can be kept and marked rather than deleted, readiness_scores.hardware_fit became nullable so "not measured" stops being reported as a score, and readiness_scores.privacy_class arrived because privacy became an ordinal class instead of a number in an average.

seed.ts shrank from 127 lines to 56 - the device seed data went with the table.

The operational rule, which matters more than any of this: against the shared Neon database, only ever run migrate deploy. Never migrate dev, which can offer to reset and would wipe everyone's data. The db:neon: * scripts in scripts/db/ exist to enforce that.

Files

File	What it is
schema.prisma	199 lines. Seven models: User, Workload, Provider, Benchmark, BenchmarkResult, ReadinessScore, SharedFinding, plus rate_limits. Carries /// doc comments explaining the non-obvious columns
seed.ts	56 lines. The provider catalogue every environment starts with. Safe to re-run: upsert() with an empty update leaves an existing row exactly as it is, so re-seeding never duplicates a provider and never clobbers a deliberate local change
migrations/	Five ordered migrations plus the lock file

Connected folders

- prisma/migrations/ - the ordered history.
- src/lib/ - prisma.ts, the single client for the process.
- src/modules/benchmark/infrastructure/repositories/ - the only code that talks to Prisma for benchmarks.
- scripts/db/ - the guards around the shared database.
- docs/internal/ - database.md, currently stale.

Referenced from

These folders point here. Each link below resolves in both directions.

- /
- prisma/migrations/20260726225537_init/
- prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/
- prisma/migrations/20260822081030_optional_email_and_session_owner/
- prisma/migrations/20260823035500_drop_devices/
- prisma/migrations/20260823165500_warmup_iteration/
- scripts/dev/
- src/
- src/app/api/v1/providers/
- src/app/api/v1/workloads/
- src/modules/benchmark/core/entities/
- src/modules/benchmark/infrastructure/repositories/

public/

public/ - statically served files

Work package: product UI.

What this folder is

Files Next.js serves verbatim at the site root. Nothing here is compiled, type-checked or linted.

What happened here

Two files were removed: `index.html` and `favicon.svg`. `index.html` was the more important removal - a static `index.html` in `public/` shadows the App Router route at `/`, so the application's real home page (`src/app/page.tsx`) was being served only when that file happened not to match. Having two definitions of the home page in two different systems is the kind of thing that works until it silently does not.

`edgepilot-live.html` was kept. It is a single 283 KB self-contained page - a standalone demo that runs with no server, no build and no database - which is useful precisely because the rest of the project needs all three.

Files

File	What it is
<code>edgepilot-live.html</code>	Self-contained demo page, 283 KB, all CSS and JS inline. Openable from disk

Connected folders

- `src/app/` - where the real routes live, and why `index.html` had to go.

scripts/ - command-line entry points

Work package: shared, by subfolder.

What this folder is

Everything invoked from `package.json` that is not `next` or `prisma` itself. The dividing line against `src/` is deliberate: nothing here is imported by the application, and nothing here runs in a request. These are operator tools - they produce evidence, guard a shared database, or set up local state.

What happened here

Three of the five subfolders are new. `db/` exists because a command aimed at the shared database could silently run against `localhost` and report success. `evaluation/` exists because the ten-case matrix the checklist asks for had no home. `dev/` exists because the scaffold endpoints did not persist, so rows had to be created from outside - and it survives now that they do, because driving a run from `curl` still needs rows.

Subfolders

Subfolder	What is in it
<code>benchmark/</code>	Real provider runs, failure captures, the clean-start log
<code>db/</code>	The guards around the shared Neon database
<code>dev/</code>	Local helpers for seeding and inspecting rows
<code>evaluation/</code>	The ten-case evaluation matrix
<code>vision-benchmark/</code>	Fixture generation, dataset validation, live and controlled runs

Connected folders

- `evidence/` - almost everything here writes into it.
- `/` - `package.json` is where these are named.
- `src/modules/` - the code these scripts drive.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/evidence/`

src/ - the application

Work package: shared, by subfolder.

What this folder is

Everything next build compiles. Five top-level divisions, and the division is the architecture:

- app/ - HTTP. Routes and pages, nothing else.
- components/ - React. What the user sees.
- modules/ - the two domain modules, each with its own core / application / infrastructure layers.
- core/, lib/, shared/ - things genuinely used by more than one module.

What happened here

Two structural simplifications, both of them removals.

src/data/ was deleted. It was a second data-access path sitting beside src/modules/*/infrastructure/repositories/, which meant the project had two answers to "where does database access live" - and two answers is the same as none.

src/core/rate-limit/ and src/core/security/ were deleted. They guarded endpoints that have no authentication and no public URL. Security machinery that protects nothing still has to be read, maintained and reasoned about by everyone who comes after.

src/core/logging/ gained SessionLog.ts, which replaced a generic logger.ts with something that has an actual policy - a redaction rule, an export format, and opt-in behaviour.

Subfolders

Subfolder	What it holds
app/	Routes and pages - the HTTP surface
components/	React components
modules/	The benchmark and vision-benchmark domain modules
core/	Cross-cutting concerns - currently the session log
lib/	The Prisma client and session ownership
shared/	Types shared across module boundaries

Connected folders

- tests/ - mirrors this tree.
- prisma/ - the schema the repositories map onto.
- docs/internal/ - architecture.md describes these boundaries in full.

Referenced from

These folders point here. Each link below resolves in both directions.

- /

tests/ - the Jest suite

Work package: shared, by subfolder, matching the module each suite covers.

What this folder is

348 tests across 33 suites, mirroring `src/` one level down. Nothing here touches a network, a real timer, a real model or a real database - every adapter takes its `fetch` and its `clock` by injection, which is what makes the suite deterministic. A benchmark test that depended on real latency would be a flaky test that also proved nothing.

What happened here

Eight files were added and six changed, taking the suite from 259 tests to 322. `tests/core/` is new; so are the suites for hardware fit, task compatibility, privacy assessment, comparison, comparison runs, the Ollama catalogue and the egress warning.

The framing is worth stating because it explains why the tests look the way they do. For a measurement tool the interesting question is not "does it work" but "**what does it refuse to say**". So most of these assert refusals: refuse to call a provider for a workload that does not exist, refuse to attribute a run to the wrong user, refuse to throw away a measurement because the database went down mid-run, refuse to declare a comparison winner inside the noise, refuse to turn a `null` into a `0`.

What this suite cannot see. Jest type-checks only what it imports. Two dev scripts and six schema-versus-interface mismatches were invisible to a green run and were caught by `next build`, which type-checks the whole project. That is why both are required checks.

One duplication is known and deliberate: `describeEgressWarning` is covered in both `egress-warning.test.ts` and `privacy-assessor.test.ts`. It is flagged rather than consolidated.

Subfolders

Subfolder	Covers
<code>benchmark/</code>	19 files - the measurement core, the adapters, the routes
<code>core/</code>	The session log
<code>dashboard/</code>	The display helpers that enforce null-honesty
<code>vision-benchmark/</code>	5 files - the vision module end to end
<code>evaluation/</code>	The ten-case artefact's shape, including that no category can vanish silently

Connected folders

- `src/` - the tree this mirrors.
- `.github/workflows/` - runs `npm test` on every push.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/`

`.github/workflows/` - **continuous integration**

Work package: integration and API surface.

What this folder is

The GitHub Actions definition that runs on every push and pull request against main and dev.

What happened here

Not modified. Worth knowing what it does and does not cover: it runs lint, tests and a build on a clean Ubuntu runner with Node 24, so it catches the class of defect that a local machine with a warm `.next` cache will not. It does **not** run the evidence scripts (`bench:run`, `vision:run:ollama`, `eval:matrix`) - those need a real provider or a real model, which a CI runner does not have.

Note that `pretest` regenerates the vision fixtures, so the CI test step implicitly proves the fixture generator works on a machine that has never seen the dataset.

Files

File	What it is
<code>ci.yml</code>	34 lines. Checkout, Node 24 with npm cache, <code>npm ci</code> , <code>npm run lint</code> , <code>npm test</code> , <code>npm run build</code>

Connected folders

- `/` - the scripts CI invokes live in `package.json`.
- `tests/` - what `npm test` runs.
- `scripts/vision-benchmark/` - the `pretest` fixture generator CI exercises on every run.

Referenced from

These folders point here. Each link below resolves in both directions.

- `.github/`

datasets/vision-benchmark/ - the reference vision dataset

Work package: vision benchmark.

What this folder is

A synthetic construction-safety dataset: 21 samples across seven labels - hardhat, safety_vest, gloves, goggles, mask, ladder, safety_cone.

Synthetic on purpose. The images are generated from a committed script rather than collected, which means the dataset has no licensing question, no privacy question, and no person in it - and it can be regenerated byte-identically on any machine. `manifest.json` records a SHA-256 so a run can prove which version of the dataset it measured.

What happened here

The dataset is unchanged. The measurement taken against it is new, and it is worth recording here because this is where a reader will come looking for it: `llava:latest` scored **66.7% accuracy** on these 21 samples - 14 correct - and **failed** the workload's own 0.80 accuracy gate. The per-class matrix closes exactly: all three hardhat samples were classified as `safety_cone`, giving hardhat an F1 of 0.0 while `safety_cone` picked up three false positives. Two gloves and two masks were also called hardhats.

The `images/` subfolder is generated and git-ignored. `npm run vision:fixtures` rebuilds it, and both `prebuild` and `pretest` call that automatically, so it is never a missing prerequisite.

Files

File	What it is
<code>manifest.json</code>	The dataset contract: schema version, dataset and workload ids, manifest version and SHA-256, title, description, source, licence, the seven labels, privacy checks, generation parameters, and the 21 sample records
<code>README.md</code>	Human description of the dataset and how to regenerate it
<code>images/</code>	Generated PNGs. Git-ignored; rebuilt by <code>npm run vision:fixtures</code>

Connected folders

- `scripts/vision-benchmark/` - `generate-fixtures.mjs` writes `images/`; `validate-dataset.ts` checks this manifest against them.
- `src/modules/vision-benchmark/infrastructure/` - `manifest-loader.ts` reads this file.
- `docs/vision-benchmark/` - the dataset card that documents provenance and licence.
- `evidence/vision-benchmark/` - the measurements taken against it.

Referenced from

These folders point here. Each link below resolves in both directions.

- `datasets/`
- `src/components/vision/`
- `src/modules/vision-benchmark/`
- `tests/vision-benchmark/`

docs/benchmark/ - **benchmark and provider layer documentation**

Work package: AI / local model & DevOps - the benchmark and provider layer.

What this folder is

The branch guide for the measurement core: what is in it, what was decided, what evidence backs each decision, and what it deliberately does not do.

What happened here

These two documents are the reason the fix list in 00-PROJECT-RECORD.md could be written with reasoning rather than guesswork - the research log records why each decision was taken **and what evidence supports it**, including the decisions to *not* measure certain things.

The guide includes a section written for the integration work package, and a statement that there is no authenticated session and ownership is derived from workloads.user_id. That statement is now historically accurate rather than currently accurate: a session id exists (see src/lib/), though the ownership rule it describes survived on its own merits - a benchmark belongs to whoever owns the workload being benchmarked.

Files

File	What it is
README.md	325 lines. Branch guide: scope, the file-by-file map of the module, the decisions, the evidence, the known gaps, and the integration notes
research-log.md	374 lines. Per-decision reasoning with official sources - the streaming contracts for Ollama, Groq and Gemini, why nanoseconds and seconds are converted where they are, and what is deliberately not measured

Connected folders

- src/modules/benchmark/ - the module documented here.
- docs/internal/ - benchmark-api.md is the wire contract these documents refer to.
- docs/local-model-setup.md - the setup guide, same work package.
- evidence/benchmark/ - the artefacts the research log cites.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/benchmark/
- src/modules/benchmark/infrastructure/providers/

docs/dashboard/ - dashboard module documentation

Work package: product UI and benchmark dashboard / integration lead.

What this folder is

One document describing the dashboard as a module: the user journey, the design decisions, and - usefully - the dependency requests it made of the other work packages.

What happened here

The document was not rewritten, and reading it now is the clearest single view of what changed in this project, because several of its open requests have since been closed:

- *"POST /api/v1/workloads currently echoes temp - *-id and does not persist"* - fixed; the route writes real rows.
- *"the dashboard detects the placeholder ids and accepts an existing database UUID instead"* - that fallback path is gone, along with the UUID field.
- *"A user_id-less benchmark history needs sessions"* - sessions exist.

The document also records the two decisions that survived unchanged and are worth keeping: one typed gateway for every request, and client validation that mirrors rather than replaces server validation.

Files

File	What it is
README.md	70 lines. The four-step user journey, the design decisions (one typed gateway, module-scoped styling under .epd, mirrored validation, accessibility), and the dependency requests to the other work packages

Connected folders

- src/components/dashboard/ - the components described.
- src/app/dashboard/ - the route and its stylesheet.
- docs/internal/ - benchmark-api.md, the contract the dashboard is written against.

Referenced from

These folders point here. Each link below resolves in both directions.

- docs/
- src/components/

docs/internal/ - maintainer documentation

Work package: integration and API surface, with `benchmark-api.md` belonging to the benchmark layer.

What this folder is

Documentation aimed at people working *on* the codebase rather than using it: architecture, database, the wire contract, and how the team works.

What happened here

`README.md` was updated to describe the current document set.

`benchmark-api.md` remains the authoritative wire contract - the dashboard's typed gateway names it in its own header as the thing it is written against, including the awkward cases: "all providers failed but the run is still evidence", and persisted: `false`.

`database.md` is **stale and known to be stale**. Line 147 still describes users owning "devices", a table that no longer exists, and the document does not mention four things that do: `shared_findings`, `users.session_id`, `benchmark_results.warmup`, and `readiness_scores.privacy_class`. It is listed in section 6 of `00-PROJECT-RECORD.md` rather than silently patched, because a documentation gap that is written down is a task and one that is not is a trap.

Files

File	What it is
<code>architecture.md</code>	401 lines. Hexagonal architecture, module boundaries, data flow, schema notes
<code>benchmark-api.md</code>	317 lines. Every endpoint of the benchmark layer with exact request and response shapes, every status code, and the eight error codes. Includes why ownership failures return 403 rather than 404
<code>database.md</code>	167 lines. Local versus shared database, migrations, seeding, conventions. Currently stale - see above
<code>team-workflow.md</code>	367 lines. Branching, issue templates, PR process and size limits, review checklist, conventional commits, daily standup, hotfix procedure
<code>project-plan.md</code>	28 lines. Internal milestones and evaluation notes
<code>Documentation.md</code>	45 lines. An earlier overview of the framework
<code>README.md</code>	27 lines. Index of this folder and pointers to the public-facing documents

Connected folders

- `prisma/` - the schema `database.md` describes.
- `src/app/api/v1/` - the endpoints `benchmark-api.md` specifies.
- `docs/benchmark/` - the module guide that refers back to the wire contract.

Referenced from

These folders point here. Each link below resolves in both directions.

- `docs/`
- `docs/dashboard/`
- `scripts/db/`
- `src/`
- `src/app/api/`

- `src/app/api/v1/benchmarks/`
- `src/modules/`

docs/pdf/ - the paginated documentation set

Work package: shared.

What this folder is

Fourteen PDFs, 241 pages, built from the 88 FOLDER.md files and 00-PROJECT-RECORD.md. Same content, laid out for reading away from a screen with the source tree open.

What happened here

Generated once and committed. They are **build output, not sources**: if a FOLDER.md changes, the PDF beside it is stale until it is regenerated, and the markdown is always the authority.

Every page carries a small grey signature in the footer, which is why the renderer is hand-rolled rather than delegated - pandoc gives no reliable control over per-page furniture.

Files

README.md lists all fourteen and what each covers. In short: 00 is the complete 114-page bundle, 01 is the project record on its own, and 02-13 are one volume per area of the tree.

Connected folders

- docs/ - the written documentation these render.
- / - 00-PROJECT-RECORD.md, the source of volume 01.

docs/vision-benchmark/ - vision module documentation

Work package: vision benchmark.

What this folder is

Seven documents covering the vision workload end to end: what the dataset is, what the workload specifies, how to execute it against a provider, what the test cases are, and how to deliver it.

What happened here

Not rewritten. Two things in here now describe behaviour that has changed underneath them and should be read with that in mind:

- The **dataset card** describes the seven-label construction dataset as the workload's dataset. That is still the reference dataset, but the module no longer requires seven labels - a user can bring their own classes from the browser.
- The **workload specification** fixes the prompt. The prompt is now built from whatever label set the dataset declares (`buildVisionPrompt(labels)`), which a deliberate failure run proved was necessary: with the fixed prompt and a custom two-class dataset the result was 0% accuracy and 100% invalid output.

Files

File	What it is
dataset-card.md	89 lines. Identity, provenance, licence, generation method, privacy checks
workload-specification.md	126 lines. What the workload sends, what it expects back, the thresholds it is graded against
provider-execution.md	106 lines. Prerequisites and how to run against Ollama or Gemini
test-cases.md	60 lines. The 48 documented cases in the automated suite
evidence-report.md	50 lines. Automated verification summary
live-modification-demo.md	51 lines. A worked demonstration of changing the workload and seeing the measurement move
pr-delivery.md	47 lines. Suggested pull-request title and body

Connected folders

- `src/modules/vision-benchmark/` - the module.
- `datasets/vision-benchmark/` - the dataset the card describes.
- `evidence/vision-benchmark/` - the runs, including the one that fails its gate.
- `scripts/vision-benchmark/` - the commands `provider-execution.md` documents.

Referenced from

These folders point here. Each link below resolves in both directions.

- `datasets/`
- `docs/`
- `src/app/vision-benchmark/`

evidence/benchmark/ - measured provider runs

Work package: benchmark and provider layer.

What this folder is

What actually happened when this application talked to a real model, and what it did when things went wrong.

What happened here

Nothing in this folder was regenerated during the changes described in 00-PROJECT-RECORD.md, and that is worth stating plainly rather than quietly fixing: **measured-ollama-llama3.2_latest.json predates the hardware-fit and privacy work**. It still records "hardwareFit": 50 - the constant every device received - and "privacyScore": 100 from the era when privacy was averaged into readiness. Read as a historical record it is accurate. Read as a description of current behaviour it is wrong, and re-running `npm run bench:run` would replace it.

Files

File	What it is
measured-ollama-llama3.2_latest.json	A real 5-iteration Ollama run: 100% success, mean latency 2,758.9 ms, p50 1,542.1 ms, mean TTFT 1,305.3 ms, 91.3 tokens/s, 623 output tokens. Carries the full environment block - CPU, cores, memory, accelerator - so the figures can be read in context
measured-ollama-llama3.2_latest.5iter-backup.json	The prior capture of the same run, kept for comparison
failure-modes.json	Timeout, fallback and error-classification behaviour driven through the real adapters and the real runner. Records whether every scenario behaved as documented, and lists mismatches if not
clean-start.log	The clean-environment setup sequence with real commands, real exit codes and real output. Explicitly un-ignored in .gitignore, because the blanket *.log rule would otherwise have swallowed a deliverable

Connected folders

- scripts/benchmark/ - run-benchmark.ts, capture-failure-evidence.ts and capture-clean-start.ts write these three.
- src/modules/benchmark/ - the code being measured.
- src/app/evidence/ - renders them.

Referenced from

These folders point here. Each link below resolves in both directions.

- docs/benchmark/
- evidence/

evidence/evaluation/ - the ten-case matrix

Work package: benchmark and provider layer.

What this folder is

One artefact: the reproducible ten-case evaluation the submission checklist asks for, covering normal, malformed, ambiguous, injection, missing-evidence and provider-failure behaviour.

What happened here

The folder is new. The behaviours it covers already existed in the unit suite, but a reviewer had to go and find them across several files. `npm run eval:matrix` now runs them in one place and writes a single readable artefact, and exits non-zero if any case misbehaves.

Every case drives **real module code** - the same zod schemas, the same normaliser, the same readiness calculator the application uses. No network call is made and no result is transcribed by hand.

The injection cases are worth understanding. EdgePilot never *acts* on model output, so the threat is not command execution; it is model output being **interpreted as a verdict**. The defence is exact matching against a closed label set, with anything else recorded as `invalid_output` rather than guessed at. Case 9 exists to assert a different rule: a missing hardware fit stays null and is never rendered as 0.

Files

File	What it is
<code>ten-case-matrix.json</code>	10 cases in 6 categories - 1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure. Current state: all 10 behaved as documented, no failed case ids. Includes the injection threat model in the artefact itself

Connected folders

- `scripts/evaluation/` - the script that writes this file.
- `src/modules/benchmark/core/services/` - the readiness calculator and assessors the cases drive.
- `src/modules/vision-benchmark/core/` - the normaliser the injection cases exercise.

Referenced from

These folders point here. Each link below resolves in both directions.

- `evidence/`
- `src/app/evaluation/`
- `src/app/evidence/`
- `tests/dashboard/`
- `tests/evaluation/`

evidence/vision-benchmark/ - image classification runs

Work package: vision benchmark.

What this folder is

Four artefacts: two controlled runs against scripted providers, and two live runs against a real model on real hardware.

Each carries the full reproducibility set - schema version, workload id and version, dataset id, manifest version **and its SHA-256**, preprocessing version, prompt version, execution mode, provider, model, git commit, start and completion timestamps, every per-sample record, the metrics, the thresholds, and whether it passed. Two runs are comparable only when those versions match, which is why they are all recorded rather than assumed.

What happened here

The two live-`*` files are new, and they are the first real measurements this project has produced. Before them the live path had never worked: the evidence writer validated filenames against a lowercase-only pattern and then built them from an ISO timestamp containing T and Z, so every live run failed at the write step.

The two live runs are the same model on the same images and differ by one change - the second was taken after a warm-up classification was added, and its median latency is **529.6 ms against the first run's 1322.1 ms**. That gap is the model load, not the model.

`.gitignore` ignores live-`*`.json by default, because live runs are machine-specific and would otherwise accumulate from every developer. The second one is explicitly un-ignored, with the reason written beside the rule.

Files

File	What it is
live-ollama-llava-latest-2026-08-23t20-03-01-476z.json	The reference measurement. llava: latest, 21 samples, 14 correct: 66.7% accuracy, macro-F1 0.667, 0% invalid output, 100% request success, median 529.6 ms, p95 721.4 ms. "passed": false - it fails its own 0.80 accuracy and 0.75 macro-F1 gates. Committed <i>because</i> it fails
the earlier pre-warm-up run	Median 1322.1 ms against the kept run's 529.6 ms. Git-ignored, so it is not in this repository - the figure is quoted here because the comparison is the point, not the file
controlled-ollama.json	Deterministic run against a scripted local provider - proves the pipeline, not the model
controlled-gemini.json	The same for the cloud provider shape

Connected folders

- datasets/vision-benchmark/ - the 21 samples these measure.
- scripts/vision-benchmark/ - run.ts writes the live files, generate-controlled-evidence.ts the others.
- src/modules/vision-benchmark/infrastructure/ - evidence-store.ts, the file that had the filename bug.
- src/app/evidence/ - renders them.

Referenced from

These folders point here. Each link below resolves in both directions.

- docs/vision-benchmark/
- evidence/
- src/modules/vision-benchmark/

prisma/migrations/ - the ordered schema history

Work packages: integration and API surface (the initial migration); benchmark and DevOps (the four newest).

What this folder is

Five timestamped folders, each holding one `migration.sql`, applied in name order. Prisma records which have run; `migration_lock.toml` pins the provider.

What happened here

Four of the five are new. Taken together they are the database half of every change described in 00-PROJECT-RECORD.md, and they were all tested against a local sandbox database first and only then applied to the shared Neon instance with `migrate deploy`.

One is a **relaxing** change and one is a **destructive** change, and the difference is worth knowing before you read them. `DROP NOT NULL` on `hardware_fit` is relaxing - it widens what the column accepts, so it cannot break a running application. `DROP TABLE "devices"` is destructive and irreversible, which is why it was rehearsed locally first.

Files

Migration	What it does
20260726225537_init/	168 lines. The original schema: users, workloads, devices, providers, benchmarks, benchmark_results, readiness_scores, rate_limits, with their indexes and unique constraints
20260822074319_nullable_hardware_fit_and_shared_findings/	Makes <code>readiness_scores.hardware_fit</code> nullable, adds <code>privacy_class</code> , and creates <code>shared_findings</code> with indexes on <code>session_id</code> and <code>created_at</code> . Nullable hardware fit is what lets "not measured" stay null instead of becoming 0
20260822081030_optional_email_and_session_owner/	Adds <code>users.session_id</code> with a unique index and makes <code>email</code> optional. This is what makes account-free row ownership possible
20260823035500_drop_devices/	31 lines. Drops two foreign keys, the <code>benchmarks_device_id_idx</code> index, the <code>benchmarks.device_id</code> column, and the <code>devices</code> table
20260823165500_warmup_iteration/	Adds <code>benchmark_results.warmup</code> <code>BOOLEAN NOT NULL DEFAULT false</code> . The discarded cold-start iteration is stored and flagged, never deleted - the arithmetic can be checked from the rows
<code>migration_lock.toml</code>	Provider lock. Not hand-edited

Connected folders

- `prisma/` - the schema these migrations produce.
- `scripts/db/` - `neon.mjs`, the only sanctioned way to run these against the shared database.
- `src/modules/benchmark/application/services/` - the runner that writes the warmup flag.

Referenced from

These folders point here. Each link below resolves in both directions.

- [prisma/migrations/20260726225537_init/](#)
- [prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/](#)
- [prisma/migrations/20260822081030_optional_email_and_session_owner/](#)
- [prisma/migrations/20260823035500_drop_devices/](#)
- [prisma/migrations/20260823165500_warmup_iteration/](#)
- [src/app/api/v1/readiness/\[id\]/ \(FOLDER.md\)](#)
- [src/app/api/v1/session-log/share/](#)

scripts/benchmark/ - **evidence producers**

Work package: benchmark and provider layer.

What this folder is

Three scripts that produce the contents of evidence/benchmark/. Between them they are the only thing in the repository that generates measured performance figures.

What happened here

run-benchmark.ts was updated alongside the runner changes - the cold-start iteration, the residency probe and the new readiness inputs all flow through it - but its contract did not change: every figure it writes carries the provenance label the adapter attached, and it deliberately does not touch the database, so it can be run before Prisma is set up.

Files

File	What it is
run-benchmark.ts	220 lines. npm run bench:run -- --provider=ollama --model=... --iterations=5. The only script that produces measured performance figures. Needs a provider that actually answers - a running Ollama or an API key
capture-failure-evidence.ts	578 lines. npm run bench:evidence:failures. Drives timeout, fallback and error-classification scenarios through the real adapters and the real runner and writes failure-modes.json. Exists separately from the tests because a green suite proves the behaviour to whoever runs it; an artefact proves it to whoever reads the pull request
capture-clean-start.ts	585 lines. npm run bench:clean-start. Runs the documented clean-environment sequence and writes down what happened, with real exit codes and real stderr. Does not summarise, does not retry silently, and exits non-zero on a failed step

Connected folders

- evidence/benchmark/ - what these write.
- src/modules/benchmark/ - the module they drive.
- docs/benchmark/ - the research log that cites their output.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/

scripts/db/ - shared-database guards

Work package: DevOps.

What this folder is

Two files whose entire purpose is to stop a command aimed at the shared Neon database from quietly running somewhere else.

What happened here

This folder is new, and it exists because of a specific failure that is easy to miss and expensive to discover late.

`dotenv -e .env.neon -- prisma migrate deploy` fails **silently** when `.env.neon` is absent. `dotenv-cli` loads nothing, Prisma falls back to its own `.env`, and a command named "neon" runs against localhost while printing "No pending migrations to apply." Nothing about the output says the target was wrong.

The inverse is worse: a file that is present but points somewhere unexpected lets a command that looks local reach a shared database, and migrate against the wrong database is not something you can recover from by reading the output afterwards.

A second, smaller problem: flags did not survive the trip. `npx dotenv -e f -- prisma migrate resolve --applied X` loses `--applied` somewhere between `npx`, `dotenv-cli` and the shell, and Prisma then complains about a flag you clearly passed.

Files

File	What it is
<code>neon.mjs</code>	126 lines. The single entry point for Prisma commands against the shared database. Reads the env file itself, sets the variables, prints the destination host before doing anything , and passes flags through intact. Backs <code>db:neon</code> , <code>db:neon:deploy</code> , <code>db:neon:seed</code> , <code>db:neon:studio</code> , <code>db:neon:status</code>
<code>require-env-file.mjs</code>	71 lines. Refuses to proceed unless <code>.env.neon</code> exists, parses, and names a host that is not local

The rule these enforce

Against the shared database, only `migrate deploy`. Never `migrate dev` - it can offer to reset, and a reset wipes everyone's data.

Connected folders

- `prisma/` and `prisma/migrations/` - what these commands apply.
- `docs/internal/` - `database.md` describes the local-versus-shared split.
- `/` - `.env.example` documents the variables; `.env.neon` itself is never committed.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`

scripts/dev/ - local development helpers

Work package: benchmark and DevOps.

What this folder is

Two throwaway-grade helpers for working against a local database. Neither is imported by the application and neither runs in CI.

What happened here

Both files broke next build during the device removal, and that is the most useful thing about them. `show-last-run.ts` read `run.deviceId` and `seed-demo-rows.ts` called `prisma.device.create`. Jest never imports either file, so the test suite stayed green while the project would not compile. This is the concrete reason `npm run build` is a required check and not an optional one.

`seed-demo-rows.ts` is now mostly obsolete by design. It existed because the workload and device endpoints were scaffold stubs that echoed `temp-*id` and never wrote anything, so an end-to-end run had to have its rows seeded from outside and their uuids pasted into the dashboard by hand. Both of those problems are gone. It survives for driving a run from curl, where there is no browser to generate a session id.

Files

File	What it is
<code>seed-demo-rows.ts</code>	75 lines. Creates the rows a benchmark run needs, for use outside the browser
<code>show-last-run.ts</code>	104 lines. Prints what the last benchmark run actually persisted, read straight from the database. The dashboard reporting persisted: <code>true</code> is the API telling you what it believes; this is independent evidence the rows are really there

Connected folders

- `prisma/` - the schema both read.
- `src/app/api/v1/workloads/` - the endpoint that made the seeding helper necessary, and then unnecessary.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`

scripts/evaluation/ - the ten-case matrix

Work package: benchmark and provider layer.

What this folder is

One script producing one artefact: the reproducible ten-case evaluation the submission checklist asks for.

What happened here

New. The behaviours were already covered by unit tests, but scattered - a reviewer had to go and find them. This runs them in one place, writes a single readable artefact, and exits non-zero if any case misbehaves, so it can sit in a pipeline.

What it proves is bounded and stated in the artefact itself: every case drives **real module code** - the same zod schemas, the same normaliser, the same readiness calculator the application uses - so the cases cannot pass against a mock that has drifted. No network call is made and no result is transcribed by hand.

Files

File	What it is
ten-case-matrix.ts	340 lines. <code>npm run eval:matrix</code> . Ten cases across six categories: 1 normal, 3 malformed, 1 ambiguous, 3 injection, 1 missing-evidence, 1 provider-failure. Case 9 asserts that a missing hardware fit stays null and never becomes 0

On the injection cases

EdgePilot never *acts* on model output - it does not execute it, follow it, or pass it to another system. The risk is narrower and easier to miss: output being read as a **verdict**. So the defence is exact matching against a closed label set, with anything that does not match recorded as `invalid_output` rather than guessed at. The threat model is written into the artefact so it can be argued with.

Connected folders

- `evidence/evaluation/` - the artefact.
- `src/modules/benchmark/core/services/` - the readiness calculator and assessors under test.
- `src/modules/vision-benchmark/core/` - the normaliser the injection cases drive.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`
- `src/app/evaluation/`
- `tests/evaluation/`

scripts/vision-benchmark/ - vision dataset and run tooling

Work package: vision benchmark.

What this folder is

Four scripts: one that builds the dataset, one that checks it, and two that run a benchmark against it - one controlled, one live.

What happened here

run.ts and validate-dataset.ts both changed to support datasets that are not the built-in seven-class one. validate-dataset.ts grew from 57 to 75 lines, mostly to stop a specific false failure: perClass was pinned to .length(VISION_LABELS.length), so a dataset with a different number of classes would run to completion and *then* be rejected by validation. Six fields had to be widened together, TypeScript interfaces and zod schemas in the same pass - widening one without the other cost three separate build round-trips.

Two operational traps are worth recording because they cost real time: npm run ... -- --flag does not always forward the flag through to the script, and node --import tsx does not load .env, so OLLAMA_VISION_MODEL was invisible and the script silently defaulted to a model that was not installed. Pass the model explicitly.

Files

File	What it is
generate-fixtures.mjs	727 lines. npm run vision:fixtures. Builds the dataset images deterministically. Called automatically by prebuild and pretest, so a build or a test run never starts from a missing dataset
validate-dataset.ts	76 lines. npm run vision:validate. Checks the manifest against the files on disk
run.ts	114 lines. npm run vision:run:ollama / vision:run:gemini / vision:run:groq. A live run against a real model, writing to evidence/vision-benchmark/
generate-controlled-evidence.ts	144 lines. npm run vision:evidence:controlled. Deterministic runs against scripted providers - proves the pipeline, not the model

Connected folders

- datasets/vision-benchmark/ - what these build and validate.
- evidence/vision-benchmark/ - what the two run scripts write.
- src/modules/vision-benchmark/ - the module they drive.
- docs/vision-benchmark/ - provider-execution.md documents these commands.

Referenced from

These folders point here. Each link below resolves in both directions.

- .github/workflows/
- datasets/
- scripts/

src/app/ - routes and pages

Work packages: product UI (pages) and integration (the API surface), with individual routes belonging to their modules.

What this folder is

The Next.js App Router. Every folder here is a URL. Nothing here holds business logic: a route parses, dispatches and picks a status code, and a page mounts a component.

What happened here

The clearest single number in this project is here. `dashboard/page.tsx` went from **692 lines to 15**. That is not deletion - the behaviour moved into `src/components/dashboard/`, where it can be tested without a request. A route file that is 692 lines long is a route file that cannot be unit tested.

`globals.css` went from 418 lines to 27 for the same reason: the dashboard's styling belongs to the dashboard, scoped under `.epd`, not to every page in the application.

`auth/` and `api/auth/` were removed - there are no accounts. A new page, `evidence/`, was added. `vision-benchmark/page.tsx` grew from 73 to 244 lines to carry the dataset upload.

Files

File	What it is
<code>layout.tsx</code>	23 lines. The root layout and metadata
<code>page.tsx</code>	37 lines. The home page
<code>globals.css</code>	28 lines. Genuinely global styling only

Subfolders

Subfolder	Route
<code>api/</code>	<code>/api/*</code> - the whole HTTP API
<code>dashboard/</code>	<code>/dashboard</code> - the benchmark journey
<code>vision-benchmark/</code>	<code>/vision-benchmark</code>
<code>evidence/</code>	<code>/evidence</code> - artefacts, rendered
<code>compare/</code>	<code>/compare</code> - two to four models against each other
<code>evaluation/</code>	<code>/evaluation</code> - the ten-case matrix, rendered
<code>history/</code>	<code>/history</code> - this session's runs, download and share
<code>setup/</code>	<code>/setup</code> - the two once-only steps for a hosted site: allow this origin in the visitor's Ollama (per-OS commands, with a connection check) and, optionally, the visitor's own Gemini/Groq keys

Connected folders

- `src/components/` - what these pages mount.
- `src/modules/` - what these routes dispatch to.
- `public/` - why a static `index.html` had to be removed from there.

Referenced from

These folders point here. Each link below resolves in both directions.

- [src/](#)

src/components/ - React components

Work packages: product UI dashboard, and vision benchmark.

What this folder is

Everything the user actually sees, in two groups by feature.

What happened here

Four files were removed from this level: `ErrorBoundary.tsx`, `Loading.tsx`, `LoadingStates.tsx` and `providers/`. Three of those were overlapping answers to the same question, and the App Router already has a convention for it - `loading.tsx` and `error.tsx` beside the route. What genuinely needed sharing is 85 lines in `dashboard/StateViews.tsx`.

`vision/` is new, and it holds a component with an unusual constraint: it must work entirely in the browser, because the images it handles are not allowed to reach the server.

Subfolders

Subfolder	What it holds
dashboard/	11 files - the benchmark journey
vision/	Both vision run panels, the browser run store, and the threshold note
compare/	The comparison application - the UI the comparison engine never had
history/	Every run this session produced, and what may leave the machine
setup/	SetupGuide - the /setup page: OLLAMA_ORIGINS instructions per OS with the site's real origin filled in, a "check the connection" probe, and the visitor's own cloud keys with a save-and-test

Connected folders

- `src/app/` - the routes that mount these.
- `docs/dashboard/` - the module guide.
- `tests/dashboard/` - the helper tests.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/`

src/core/ - cross-cutting concerns

Work package: benchmark and provider layer.

What this folder is

Things used by more than one module and owned by none of them. Exactly one concern lives here now: the session log.

What happened here

Two whole subfolders were removed and one file was replaced.

rate-limit/ and security/ guarded endpoints that have no authentication and no public URL. Security machinery that protects nothing is not neutral - it still has to be read, maintained and reasoned about by everyone who comes after, and it implies a threat model the application does not actually have.

logging/logger.ts was replaced by logging/SessionLog.ts, which is the difference between a logging utility and a logging **policy**: a redaction rule, an export format, and opt-in behaviour.

Subfolders

Subfolder	What it holds
logging/	The session activity log and its store
quota/	cloudRunQuota.ts - a small in-memory, per-session cap on cloud vision runs that spend the SERVER's key; a visitor using their own key is not capped. Best-effort, not a security boundary (per process on a serverless host)

Connected folders

- src/app/api/v1/session-log/ - the endpoints that expose it.
- tests/core/ - the suite.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/

src/lib/ - process-level singletons

Work package: integration and API surface; `sessionOwner.ts` belongs to the benchmark layer.

What this folder is

Two files, both answering a question that has exactly one correct answer per process: which Prisma client, and who owns this row.

What happened here

`sessionOwner.ts` is new, and it closed a real data leak.

Two routes previously attributed every row they created to a single hardcoded user, `local-sandbox@edgepilot.invalid`, with a comment saying session handling did not exist yet. On one laptop that is harmless. The moment the application is hosted it is a leak: GET would hand every visitor every other visitor's workloads, because they would all be the same user.

The replacement **fails closed**. A request with no session id gets a 400. It does not fall back to a shared user - that fallback is exactly the leak, and it fails silently, because the caller gets a 200 and someone else's data.

Files

File	What it is
<code>prisma.ts</code>	33 lines. The single <code>PrismaClient</code> for the process, cached on <code>globalThis</code> . Without the cache, a new client per hot reload exhausts the database connection limit within a few edits
<code>sessionOwner.ts</code>	76 lines. <code>sessionIdOf(request)</code> , <code>resolveOwnerId(sessionId)</code> which upserts a <code>User</code> , and <code>missingSession()</code> which returns the 400

Connected folders

- `src/components/dashboard/` - `session.ts`, the browser half that generates the id.
- `src/app/api/v1/workloads/` - the route that used to attribute everything to one user.
- `prisma/` - the `users.session_id` column and the migration that added it.

Referenced from

These folders point here. Each link below resolves in both directions.

- `docs/benchmark/`
- `src/`
- `src/app/api/v1/`
- `src/modules/benchmark/infrastructure/repositories/`

src/modules/ - the domain modules

Work packages: benchmark, and vision benchmark.

What this folder is

Two modules, each with the same three layers, each able to change without the other:

- **core/** - domain logic. Pure. Imports nothing from infrastructure/.
- **application/** - use cases and the typed envelopes they return.
- **infrastructure/** - adapters. HTTP clients, Prisma, the filesystem.

What happened here

modules/device/ was removed entirely. It held a self-reported hardware profile that no measurement ever consumed - the user typed how much RAM they had and nothing checked it. Once hardware fit became a VRAM residency reading from the runtime, the typed value had no consumer left and could only disagree with reality.

The benchmark module grew by eight core services and two infrastructure adapters. Every one of them replaced something that was previously a constant, an assumption, or nothing at all.

The two modules deliberately **duplicate** a small amount of transport code rather than sharing it - `http.ts` exists in both, with a comment saying the copy is intentional, because the two modules must be able to change independently. A shared helper here would couple the vision workload's release cycle to the benchmark layer's.

Subfolders

Module	What it does
benchmark/	Text and code generation benchmarks across local and cloud providers
vision-benchmark/	Image classification benchmarks

Connected folders

- `src/app/api/v1/` - the routes that dispatch here.
- `tests/` - mirrors both modules.
- `docs/internal/` - `architecture.md`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/`
- `src/`
- `src/app/`

src/shared/ - types crossing module boundaries

Work package: integration and API surface.

What this folder is

The scaffold's shared type definitions and zod schemas.

What happened here

types/device.ts was removed with the device module. The rest is largely unchanged, and its role has narrowed in a way worth understanding: the benchmark module now defines its own richer measurement envelope in application/dtos/BenchmarkMeasurement.ts, and the dashboard imports **that** rather than these. These remain the scaffold contract and the shape the original endpoints were written against.

Subfolders

Subfolder	What it holds
types/	Benchmark, provider and workload types with their zod schemas

Connected folders

- src/modules/benchmark/application/dtos/ - the superset that supersedes these for measurement.
- src/app/api/v1/ - the routes that validate with them.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/

tests/benchmark/ - the measurement core under test

Work package: benchmark and provider layer.

What this folder is

Nineteen files covering the provider adapters, the runner, the use cases, the core services and the API route.

What happened here

Seven files are new, all of them covering services that did not exist before: hardware fit, task compatibility, privacy assessment, comparison planning, comparison runs, the Ollama catalogue and the egress warning.

`run-benchmark.test.ts` changed most among the existing files (73 lines differing) - it uses the **real** `BenchmarkRunner` over scripted providers rather than a stubbed runner, so the envelope that comes out is the one the API actually returns, and the cold-start change moved through it.

One fix in here is worth calling out because of how it was made rather than what it was: a run of three iterations was reporting "4/4 iterations succeeded", because the success chain was counting the discarded warm-up. The fix went into the runner, not into the assertion.

Files

File	What it covers
<code>helpers.ts</code>	The test doubles: fake providers, a stepped clock, a frozen clock, SSE and NDJSON response builders, connection-refused and hanging fetch stubs
<code>benchmark-runner.test.ts</code>	When the runner falls back. The rule: fall back only when the provider failed for a reason another provider could plausibly not share
<code>run-benchmark.test.ts</code>	The use case - ownership refusals, persistence failure mid-run, and the full envelope
<code>benchmarks-route.test.ts</code>	Parse, dispatch, status code. Above all: refuse a malformed body before anything constructs a provider, because a provider call costs money, quota and up to five minutes
<code>benchmark-measurement.test.ts</code>	What the summary refuses to report. This is where a number would be invented if anywhere
<code>benchmark-config.test.ts</code>	Environment validation and the server-side guard
<code>hardware-fit.test.ts</code>	VRAM residency, spill share, and the states between
<code>task-compatibility.test.ts</code>	The asymmetric rule - vision can do text, text cannot do vision, embedding neither
<code>privacy-assessor.test.ts</code>	Privacy class, disqualifiers, egress classification
<code>egress-warning.test.ts</code>	The pre-flight warning, including that it does not carry the prompt text
<code>comparison.test.ts</code>	Planning and reporting: overlapping ranges must not produce a winner
<code>run-comparison.test.ts</code>	The comparison use case, including sequential execution for local entrants
<code>visitor-keys.test.ts</code>	Header parsing for a visitor's own cloud keys: trimmed, shape-checked, laid over the server's per provider
<code>recorded-provider.test.ts</code>	A browser-recorded run through the runner: cold start discarded, browser residency readings used, no cloud fallback; and the request schema's recording rules
<code>ollama-catalog.test.ts</code>	Runtime status and the installed-model list
<code>cloud-catalog.test.ts</code>	Gemini and Groq model lists: filtering, pagination, key-in-header, and each failure message
<code>ollama-provider.test.ts</code>	NDJSON streaming, nanosecond conversion, TTFT
<code>gemini-provider.test.ts</code>	SSE parsing, <code>usageMetadata</code> , header-carried key

File	What it covers
groq-provider.test.ts	OpenAI-compatible SSE, stream_options.include_usage, completion_time in seconds
demo-provider.test.ts	The one promise that makes the demo adapter acceptable: nothing it produces can be mistaken for a measurement
provider-errors.test.ts	The published error vocabulary. This is what stops the code and docs/local-model-setup.md drifting apart
provider-registry.test.ts	Which adapters are usable in a given environment, and in what order

Connected folders

- src/modules/benchmark/ - the module under test.
- src/app/api/v1/benchmarks/ - the route benchmarks-route.test.ts drives with the container mocked.
- docs/local-model-setup.md - the error-code table provider-errors.test.ts pins.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/app/api/v1/comparisons/
- src/app/api/v1/local-runtime/
- src/modules/benchmark/application/
- src/modules/benchmark/application/dtos/
- src/modules/benchmark/application/services/
- src/modules/benchmark/application/use-cases/
- src/modules/benchmark/core/
- src/modules/benchmark/core/ports/
- src/modules/benchmark/core/services/
- src/modules/benchmark/infrastructure/
- src/modules/benchmark/infrastructure/providers/
- tests/

tests/core/ - cross-cutting concerns under test

Work package: benchmark and provider layer.

What this folder is

One suite, covering the session log.

What happened here

New, alongside `src/core/logging/`. The tests exist mainly to pin the redaction rule, which is the part of the session log that would be easiest to relax by accident: prompt **content** is never recorded by default, only a length and a digest. A log that wrote out the same prompt the egress warning had just flagged would have defeated its own warning.

They also pin the opt-in behaviour: a request without a session header is not logged anonymously and not logged to a default bucket - it is not logged at all.

Files

File	What it covers
<code>cloud-run-quota.test.ts</code>	The per-visitor cap on cloud vision runs made with the server's key: the allowance, the hour window, and session separation
<code>session-log.test.ts</code>	224 lines. Redaction, prompt digests, the share payload and its consent statement, and the in-memory store's bounds

Connected folders

- `src/core/logging/` - the code under test.
- `src/app/api/v1/session-log/` - the endpoints that expose it.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/core/`
- `src/core/quota/`
- `tests/`

tests/dashboard/ - display helpers under test

Work package: product UI.

What this folder is

One suite, and it defends one rule.

What happened here

The wire contract makes every aggregate nullable on purpose, because reporting 0 ms for "we never got an answer" is the single most misleading number this system could produce. These tests pin the helpers that stop a null becoming a 0 on screen - which is the last place the rule could be broken, after the data layer has got it right.

Files

File	What it covers
format.test.ts	77 lines. fmtMs, fmtNum, fmtPct, fmtElapsed, describeErrorCode, isUuid, isPlaceholderId - each asserted to render "not measured" rather than a zero

Connected folders

- src/components/dashboard/ - format.ts, the file under test.
- evidence/evaluation/ - case 9 of the ten-case matrix asserts the same rule one layer down, in the data.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/components/
- tests/

tests/evaluation/ - the matrix artefact under test

Work package: benchmark and provider layer.

What this folder is

One suite, pinning the shape of evidence/evaluation/ten-case-matrix.json.

What happened here

New. The matrix is now rendered as a page, which means its shape is a contract rather than a private detail - and the failure it guards against is a quiet one: a category the page cannot render would otherwise **vanish from the list silently**, leaving a matrix that looks complete while showing nine cases.

That is the same class of defect as reporting 0 for "not measured": not a crash, just a result that is wrong in a way nobody notices.

Files

File	What it covers
ten-case-matrix.test.ts	134 lines. Case count, the six categories, and that every category the artefact declares can actually be rendered

Connected folders

- evidence/evaluation/ - the artefact.
- scripts/evaluation/ - what writes it.
- src/app/evaluation/ - the page that renders it.

Referenced from

These folders point here. Each link below resolves in both directions.

- tests/

tests/vision-benchmark/ - the vision module under test

Work package: vision benchmark.

What this folder is

Five suites covering the vision workload from schema to provider.

What happened here

`vision-execution.test.ts` changed most (86 lines differing), because the warm-up classification changed the shape of an execution.

That change is a useful record of how a small feature goes wrong. The warm-up consumed a deterministic test clock, so the timings moved - fixed by calling `provider.classify()` directly rather than the timed wrapper. It fired on cloud providers, which meant a billable call and one extra image sent to a third party - fixed by restricting it to local providers and adding an explicit `warmUp?: boolean`. And it ate the first of the scripted `outputs.shift()` responses in two existing tests - those two now opt out, with a comment saying why, and two new tests cover the warm-up behaviour properly.

`vision-api.test.ts` changed to accept datasets whose label set is not the built-in seven.

Files

File	What it covers
<code>vision-benchmark.test.ts</code>	356 lines. The workload end to end: metrics, thresholds, evidence shape
<code>vision-execution.test.ts</code>	362 lines. The executor, including warm-up behaviour and its opt-out
<code>vision-providers.test.ts</code>	The Ollama, Gemini and Groq vision adapters
<code>vision-infrastructure.test.ts</code>	204 lines. Image processing, manifest loading, the evidence store
<code>vision-api.test.ts</code>	78 lines. The <code>/api/v1/vision-benchmarks</code> route

Connected folders

- `src/modules/vision-benchmark/` - the module under test.
- `datasets/vision-benchmark/` - the fixtures pretest regenerates before these run.
- `src/app/api/v1/vision-benchmarks/` - the route.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/vision-benchmark/application/`
- `src/modules/vision-benchmark/infrastructure/`
- `tests/`

`prisma/migrations/20260726225537_init/`

`prisma/migrations/20260726225537_init/`

Work package: see `prisma/migrations/FOLDER.md`.

What this folder is

One Prisma migration: the original schema - eight tables, their indexes and unique constraints. Everything else in this folder is a delta against it.

Files

File	What it is
<code>migration.sql</code>	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- `prisma/migrations/` - the full ordered history and what each migration changes.
- `prisma/` - the resulting schema.

`prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/`

`prisma/migrations/20260822074319_nullable_hardware_fit_and_shared_findings/`

Work package: see `prisma/migrations/FOLDER.md`.

What this folder is

One Prisma migration: makes hardware fit nullable, adds the privacy class column, and creates the `shared_findings` table.

Files

File	What it is
<code>migration.sql</code>	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- `prisma/migrations/` - the full ordered history and what each migration changes.
- `prisma/` - the resulting schema.

prisma/migrations/20260822081030_optional_email_and_session_owner/

prisma/migrations/20260822081030_optional_email_and_session_owner/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: adds a unique session id to users and makes email optional, so a row can be owned without an account.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260823035500_drop_devices/

prisma/migrations/20260823035500_drop_devices/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: removes the devices table and the benchmarks.device_id column, with their constraints and index.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

prisma/migrations/20260823165500_warmup_iteration/

prisma/migrations/20260823165500_warmup_iteration/

Work package: see prisma/migrations/FOLDER.md.

What this folder is

One Prisma migration: adds the warmup flag to benchmark_results so a discarded cold-start iteration is marked rather than deleted.

Files

File	What it is
migration.sql	The SQL Prisma applies for this step. Never edited after it has been applied anywhere

Connected folders

- prisma/migrations/ - the full ordered history and what each migration changes.
- prisma/ - the resulting schema.

src/app/api/

src/app/api/ - the HTTP API

Work package: integration and API surface.

What this folder is

Route handlers. One version namespace, v1.

What happened here

api/auth/ was removed along with the auth pages. Everything else lives one level down.

Subfolders

Subfolder	What it is
v1/	The versioned API - nine endpoints

Connected folders

- src/app/api/v1/ - the endpoints.
- docs/internal/ - benchmark-api.md is the authoritative wire contract.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/compare/ - the comparison route

Work package: product UI and benchmark dashboard.

What this folder is

/compare. A 14-line route file that mounts CompareApp.

What happened here

New, and it closed a gap that had been in the known-gaps list as *"works, nothing calls it"*: /api/v1/comparisons had a complete backend, a planner that decides whether entrants may be compared at all, and a report that refuses to name a winner inside the noise - and no user interface whatsoever.

The route is thin on purpose. All 899 lines of behaviour live in src/components/compare/, where they can be tested without a request.

Hosted-mode pass. Nothing at this level changed; see src/components/compare/ for the browser-side measurement of Ollama entrants.

Files

File	What it is
page.tsx	14 lines. Metadata and a mount point

Connected folders

- src/components/compare/ - the application.
- src/app/api/v1/comparisons/ - the endpoint it drives.
- src/modules/benchmark/core/services/ - ComparisonPlanner and ComparisonReport, which decide what may be claimed.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/dashboard/ - the benchmark dashboard route

Work package: product UI and benchmark dashboard.

What this folder is

The /dashboard route: a page, a loading state, an error state, and the module's stylesheet.

What happened here

page.tsx went from **692 lines to 15**. Everything it used to do now lives in src/components/dashboard/, which is the difference between behaviour that can be unit tested and behaviour that can only be tested through a request. dashboard.css grew by 45 lines - the installed-models panel and the cold-start row in the results needed styling. The stylesheet is entirely scoped under .epd. That scoping is why src/app/globals.css could shrink from 418 lines to 27: the dashboard's design system stopped leaking into the home page and the vision dashboard.

Hosted-mode pass. dashboard.css gained the /setup page's styles (steps, command block, key rows) and btn-row-baseline, a row that aligns buttons with a labelled control's bottom edge.

Files

File	What it is
page.tsx	16 lines. Metadata and a mount point
dashboard.css	614 lines. The module-scoped design system. Light and dark, reduced-motion, focus-visible, and a colour-vision-deficiency validated provider palette in which colour follows the provider entity, never rank or state
loading.tsx	16 lines. The App Router's route-level loading convention
error.tsx	30 lines. The route-level error boundary - which is why the team's three separate loading and error-boundary components were not needed

Connected folders

- src/components/dashboard/ - where the 692 lines went.
- docs/dashboard/ - the module guide.
- src/app/ - why globals.css is 27 lines.

src/app/evaluation/ - the ten-case matrix, rendered

Work package: product UI and benchmark dashboard.

What this folder is

/evaluation. Renders evidence/evaluation/ten-case-matrix.json as a page: all ten cases, the request that produced each one, and the behaviour observed.

What happened here

New. The matrix existed as an artefact a reviewer had to open a JSON file to read; this makes it readable in a browser without losing the detail that makes it evidence.

The three prompt-injection cases are shown in full rather than summarised as a count. That is the point of them: the defence is exact matching against a closed label set with anything else recorded as `invalid_output`, and a reader can only judge whether that is sufficient by seeing what was actually sent.

`categories.ts` exists so a category the page cannot render fails loudly instead of vanishing from the list - a silently dropped case would make the matrix look complete when it was not.

Files

File	What it is
page.tsx	237 lines. The ten cases with their requests and observed behaviour
categories.ts	42 lines. The six category labels and their descriptions

Connected folders

- evidence/evaluation/ - the artefact rendered.
- scripts/evaluation/ - what writes it.
- tests/evaluation/ - the suite that pins the artefact's shape.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/evidence/ - the evidence page

Work package: DevOps.

What this folder is

The /evidence route: everything in evidence/ rendered in the browser, so a reviewer can read the artefacts without opening a terminal.

What happened here

New. Two things about it are deliberate.

It leads with the limits. Every artefact is introduced by its own `what_this_proves` and `what_this_does_not_prove` fields before a single number appears. An evidence page that opened with a score would be a marketing page.

Its file reads use literal path segments. The first version built paths with a dynamic `path.join(process.cwd(), root)` inside a loop, which defeats Next's file tracing - the tracer cannot see which files are needed, so it conservatively traced the entire project into the standalone bundle. Rewritten as two explicit calls with literal segments.

Files

File	What it is
page.tsx	286 lines. Reads evidence/benchmark/ and evidence/evaluation/ at request time and renders each artefact with its claims and its caveats

Connected folders

- evidence/ - what this page reads.
- evidence/evaluation/ - the ten-case matrix it renders.
- scripts/ - what writes the files.

Referenced from

These folders point here. Each link below resolves in both directions.

- evidence/benchmark/
- evidence/vision-benchmark/
- src/app/

src/app/history/ - the session history route

Work package: shared.

What this folder is

/history. The route, its header, and the shared navigation; the history itself is a client component because it lives in the visitor's browser.

What happened here

New. Three kinds of run were being produced and then dropped: the dashboard held a benchmark in React state until you navigated away, a comparison - the most expensive thing this application does - survived nowhere at all, and vision runs were kept but only visible on their own page.

The page is organised **by where data goes, not by what kind of data it is**. One section never leaves the machine; the other only leaves on an explicit confirmation. A history page that mixed "saved locally" with "sent to the maintainers" into one list would make the second easy to do by accident, which is the exact thing the share flow was built to prevent.

The header lives here rather than in the component because this is a destination people arrive at from a run and need a way back out of. Without it the page was a dead end - and auditing that turned up the same problem on the vision page, which had only a "home" link.

Files

File	What it is
page.tsx	59 lines. Metadata, the epd header with the full nav row, and the mount point

Connected folders

- src/components/history/ - the work.
- src/components/vision/ - runHistory.ts, where all three kinds of run are stored.
- src/app/api/v1/session-log/ and share/ - the export and the consent-gated upload.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/setup/ - the setup route

Work package: product UI and benchmark dashboard.

What this folder is

/setup. The two things a visitor does once so the hosted site works with their own machine and their own accounts: allow this site's origin in their Ollama, and (optionally) paste their own Gemini and Groq keys.

What happened here

New, and it exists because of hosting. On a developer's laptop the page and Ollama share one machine, and Ollama trusts localhost by default. On Vercel the page is served from another origin, and Ollama - correctly - refuses any website it was not told to trust. That refusal cannot be undone by the page; it is a one-time setting on the visitor's side, per operating system. This route gives it the room it needs: OS tabs, the exact command with the site's real origin filled in, a copy button, and a "check the connection" probe.

The keys section is the visitor's answer to a shared server key: theirs are stored in their browser, sent with each of their requests, used for that call, and never written down on the server.

Files

File	What it is
page.tsx	Metadata, the epd header with the full nav row, and the mount point for SetupGuide

Connected folders

- src/components/setup/ - the work.
- src/modules/benchmark/infrastructure/ - browser-ollama.ts (the probe) and visitor-keys.ts (the server half of the keys).

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/vision-benchmark/ - the vision dashboard route

Work package: vision benchmark.

What this folder is

The /vision-benchmark route: the image-classification workload, its results, and the bring-your-own-dataset panel.

What happened here

Grew from 73 to 244 lines, almost all of it to host the dataset upload. That feature is the reason the page has a client component in it at all.

The design rule the page has to respect: an uploaded image **never reaches the EdgePilot server**. It is resized and hashed in the tab, and classified by talking from the tab straight to the user's own Ollama on localhost. So the page mounts a browser image processor and a browser provider rather than posting files to an endpoint.

Files

File	What it is
page.tsx	The workload description, the reference table, and the mount points for both run panels
actions.ts	The server action that runs the built-in dataset on a CLOUD provider (Gemini or Groq) - the local run happens in the browser, see components/vision/builtInDataset.ts. Takes the provider, model, session id and the visitor's own keys as arguments, since a server action carries no request headers. Server-key runs are capped per visitor per hour (core/quota). Builds the request exactly as the CLI does; writes nothing server-side
types.ts	The action's result shape. Separate because a file marked 'use server' may only export async functions - exporting an interface from one is a build error

Connected folders

- src/components/vision/ - DatasetUpload.tsx, the upload panel.
- src/modules/vision-benchmark/ - the module.
- src/components/dashboard/ - VisionHandoff.tsx, how a user arrives here from the benchmark dashboard.
- docs/vision-benchmark/ - the workload specification.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/
- src/core/quota/

src/components/compare/ - the comparison application

Work package: product UI and benchmark dashboard.

What this folder is

One 899-line client component driving `/api/v1/comparisons`: pick two to four entrants, send one prompt, read a per-dimension verdict.

What happened here

New, and it is the surface that made the comparison engine usable. Three things in it are worth knowing because they show it was written against this codebase rather than in spite of it.

It mirrors the server's validation exactly - 4,000-character prompt cap, 1 to 20 iterations, at most four entrants - which is the convention the dashboard docs state: client validation mirrors, never replaces.

It understands the measurement design. It tells the user "{iterations} measured + 1 discarded cold start" and "two local entrants run one after the other", rather than just calling the endpoint. Those are the N+1 warm-up and ComparisonPlanner's sequential rule, surfaced rather than hidden.

It renders "no winner" as a result. Every dimension carries an honesty code, and a row where the observed ranges overlap says so instead of showing a blank. That is the whole argument of ComparisonReport; a UI that rendered it as an empty cell would have quietly undone it.

It also refuses the same model twice, mirroring the planner's refusal - the labels are `provider + model` and the report's tally is keyed by them, so a duplicate collides onto one key and merges two entrants into one verdict.

Hosted-mode pass. Ollama entrants are measured in the browser, one after another, before the request is sent, with the progress line naming the model being measured; cloud entrants are measured by the server. The model field is a list for every provider - the visitor's installed models for Ollama, the vendor's list for Gemini and Groq - and the placeholder cannot be submitted. The second entrant defaults to the first configured cloud provider (Gemini before Groq) so the page opens as the local-versus-cloud pairing the app exists for, and never overrides a choice already made.

Files

File	What it is
CompareApp.tsx	899 lines. Entrant form, egress preview, plan, per-dimension verdicts, and the session-log controls

Connected folders

- `src/app/compare/` - the route.
- `src/app/api/v1/comparisons/` - the endpoint.
- `src/modules/benchmark/core/services/` - ComparisonPlanner, ComparisonReport.
- `src/components/vision/` - `runHistory.ts`, where a completed comparison is stored.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/components/`

src/components/dashboard/ - the benchmark journey

Work package: product UI and benchmark dashboard / integration lead.

What this folder is

Eleven files implementing the four-step user journey: workload, provider and model, run, results. This is where the 692 lines that used to be in `app/dashboard/page.tsx` now live, and the move is what makes them testable.

What happened here

The largest change in the folder is a **deletion**. `SetupPanel.tsx` went from 322 lines to 201 - 379 lines differing - because the UUID field and everything around it went away. Users were being asked to paste a workload uuid by hand, because the endpoint that should have created one did not persist. Once `POST /api/v1/workloads` wrote real rows, the field, the placeholder-id detection and the "this was not saved" warning all became dead weight.

The identity that replaced it is `session.ts`, and the important property is that it is **never rendered**. A user should no more be asked to see their own row-ownership key than to type their own cookie.

`InstalledModels.tsx` and `VisionHandoff.tsx` are new. The installed-models panel is always visible and filters by step: on step 1 models that cannot serve the chosen task are greyed out; from step 2 only models that will work are offered at all.

The input and output format fields became read-only derived text - an informant, not a control. They describe what the chosen task sends and returns; they were never things a user could meaningfully change.

Hosted-mode pass. `InstalledModels`, `ProviderPanel` and `DashboardApp` ask the visitor's own Ollama from the browser instead of the server; `ProviderPanel` lists Gemini and Groq models from the vendors (`getProviderModels`) rather than a free-text box, and every placeholder option is disabled; `DashboardApp` clears the model when the provider changes so a Groq name is never submitted to Gemini; `RunPanel` says where the run happens. `apiKeys.ts` is new, and `api.ts` attaches the visitor's keys to every call and, for Ollama, measures in the tab before posting the recorded result.

Files

File	What it is
<code>DashboardApp.tsx</code>	204 lines. The step machine and shared state
<code>SetupPanel.tsx</code>	202 lines. Step 1 - the workload. No uuid field any more
<code>ProviderPanel.tsx</code>	295 lines. Step 2 - the live catalogue. Unconfigured providers are visible but not selectable
<code>RunPanel.tsx</code>	316 lines. Step 3 - prompt, iterations, elapsed feedback. Tells the user their iteration count will be raised by one for the discarded cold start
<code>RunResults.tsx</code>	414 lines. Step 4 - per-iteration table with provenance badges, null-honest summary tiles, the fallback chain, readiness and its stored breakdown, evidence, assumptions, limitations, JSON export, and the cold-start row
<code>InstalledModels.tsx</code>	245 lines. The always-visible model panel, with <code>fitFor</code> deciding what is offered at each step
<code>VisionHandoff.tsx</code>	88 lines. The route into the vision dashboard
<code>StateViews.tsx</code>	85 lines. <code>EmptyState</code> , <code>ErrorState</code> , <code>LoadingState</code> - what replaced three separate top-level components
<code>apiKeys.ts</code>	The visitor's own cloud keys, in this browser's <code>localStorage</code> ; <code>apiKeyHeaders()</code> attaches them to every API call. Components get only "is one set", never the value

File	What it is
<code>api.ts</code>	The one typed gateway. For Ollama, <code>getLocalRuntime</code> and <code>runBenchmark/runComparison</code> talk to the visitor's own runtime from the tab (see <code>infrastructure/browser-ollama.ts</code>) and send the measurements to the server to be scored - the server cannot reach a visitor's machine once hosted. Every request goes through it, so components render exactly one error shape. Types come from the benchmark module's DTOs via <code>import type</code> , erased at build time, so the dashboard cannot drift from the measurement envelope without a compile error
<code>format.ts</code>	86 lines. Display helpers whose job is that no component ever turns a null into a 0
<code>session.ts</code>	92 lines. The browser's own durable id in <code>localStorage</code> , sent as <code>x-edgepilot-session</code> , never displayed. <code>localStorage</code> and not <code>sessionStorage</code> , because a workload registered in one tab has to still be yours in the next tab, and tomorrow

Connected folders

- `src/app/dashboard/` - the route and the `.epd-scoped` stylesheet.
- `src/app/api/v1/` - what `api.ts` talks to.
- `src/lib/` - `sessionOwner.ts`, the server half of `session.ts`.
- `src/modules/benchmark/application/dtos/` - the types `api.ts` imports.
- `tests/dashboard/` - `format.test.ts`.
- `docs/dashboard/` - the module guide, including the requests this folder made of the other work packages.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/local-runtime/`
- `src/app/api/v1/providers/`
- `src/components/setup/`
- `src/app/api/v1/readiness/[id]/` (FOLDER.md)
- `src/app/api/v1/session-log/`
- `src/app/api/v1/workloads/`
- `src/app/vision-benchmark/`
- `src/components/`
- `src/components/history/`
- `src/core/logging/`

src/components/history/ - everything this session produced

Work package: shared.

What this folder is

One client component rendering three run tables and the session-log controls.

What happened here

New. Three tables, one per kind of run - text and code, comparisons, vision - each with per-row download, download-all, select-some, and clear. Plus the activity log: download it, or share it through the existing consent flow.

Two rules it exists to hold:

The preview is the payload. The share section renders `would_send` directly, and the consent statement and disclosure lines come from the server rather than being retyped here. A preview that described the upload instead of being it would drift, and a drifted preview is worse than none.

A dataset you supplied is download-only. Runs against the built-in dataset may be shared; runs against your own images may not, because their sample ids are your filenames, their labels are your folder names, and the outputs are a model describing your pictures. The table marks every row `built-in` or `yours` so this is visible before anything is clicked.

The session id is never rendered, the same rule `components/dashboard/session.ts` states.

Hosted-mode pass. The database panel that sat at the bottom of the page was removed with its endpoint - a debugging aid on a user-facing page, and one that told any visitor there was a Postgres to talk to. The vision-runs block was brought in line with the other two (per-row download, download-all, clear; no checkbox selection). The text-and-code table gained a **Measured in** column: `this` browser for an Ollama run the tab made against the visitor's own machine, `server` for a cloud run.

Files

File	What it is
<code>SessionHistory.tsx</code>	Three run tables, each with per-row and bulk download and clear, the log export, and the consent-gated share

Connected folders

- `src/app/history/` - the route and its header.
- `src/components/vision/` - `runHistory.ts`, the store all three tables read.
- `src/components/dashboard/` - `api.ts` supplies the share preview and upload functions.
- `src/core/logging/` - what the activity log is.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/components/`

src/components/setup/ - the setup guide

Work package: product UI and benchmark dashboard.

What this folder is

One client component, SetupGuide, mounted by /setup.

What happened here

Two sections. **Ollama:** the operating system is guessed from the user agent so the right tab opens, the site's real origin is read from `window.location` and written into the command, and a button probes the visitor's Ollama from the tab and reports "connected, N models" or the remedy. A passed check is remembered in that browser, so a returning visitor is told there is nothing to redo. On a page served from localhost the section says so and skips the steps.

Keys: two password fields with show/hide, Save, and Save & test - the test lists the vendor's models with that key, so a wrong key is caught before a run spends it. Values go to `localStorage` through `apiKeys.ts`; nothing that renders is ever given the key itself.

The command shown uses the site's own origin, not *. Allowing every website would hand the visitor's GPU to any page they open; the note beneath says how to widen it if they want to.

Files

File	What it is
SetupGuide.tsx	The page body: OS tabs and per-OS commands, the connection check, and the two key fields

Connected folders

- `src/app/setup/` - the route.
- `src/components/dashboard/` - `apiKeys.ts` and `api.ts`, where the keys are stored and attached.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/components/`

src/components/vision/ - bring your own dataset

Work package: vision benchmark.

What this folder is

Four files. One component takes your own images and never lets them leave the machine; another runs the dataset the project ships with; the other two are the store behind them and the note that says what "passed" means.

What happened here

New. A user picks a folder laid out one directory per class (`webkitdirectory`), and the component measures a real model against their own photographs without a single image reaching the EdgePilot server. Resizing and hashing happen in the tab; classification goes from the tab straight to the user's own Ollama on localhost.

That is not a convenience decision. Benchmarking confidential data is this project's stated non-goal, and sending images to a server to be resized would have contradicted the warning the rest of the application gives.

Four things had to be got right, and three of them were got wrong first:

- **LabelOf requires three path parts.** With two, a loose file sitting at the chosen root adopted the root folder's own name as its class.
- **The prompt comes from the dataset.** A deliberate failure run proved it: with the built-in seven-label prompt and a custom two-class dataset, the result was 0% accuracy and 100% invalid output.
`buildVisionPrompt(labels)` builds the closed label set from whatever the user brought. Re-run properly with `red_panel/green_panel`: 100%, passed.
- **The metrics schema had to be widened.** `perClass` was pinned to exactly seven entries, so a different class count ran to completion and *then* failed validation.
- **Four declarations block the run** - SPDX licence text plus three privacy confirmations. A dataset with no stated licence and no confirmation of what is in the images is not one anybody should publish a metric from.

Files

File	What it is
<code>RunBuiltInDataset.tsx</code>	Runs the shipped 21-image dataset from the page, on Ollama (this computer), Gemini or Groq, and shows the result as a table with the cell that missed its threshold highlighted. Model lists come from each provider; the cloud lists are narrowed to models that accept an image
<code>builtInDataset.ts</code>	The Ollama run, performed IN THE BROWSER: fetches the manifest and images from <code>/api/v1/vision-benchmarks/dataset</code> , runs the shared executor against the visitor's own Ollama, then reads residency for hardware fit. Hosted, the server has no Ollama, so this is the only way a local vision run can happen
<code>runHistory.ts</code>	Every run this browser has made - vision, text and code, and comparisons - each capped at 20. Every access is wrapped, because <code>localStorage</code> throws outright in some privacy modes rather than returning null
<code>ThresholdNote.tsx</code>	States the gate: accuracy, macro F1, invalid-output and success-rate bars, rendered from <code>DEFAULT_VISION_THRESHOLDS</code> rather than typed in, so the copy cannot drift from the rule the evaluator applies
<code>DatasetUpload.tsx</code>	403 lines. Folder-per-class picker, <code>LabelOf</code> , the four blocking declarations, prompt construction from the dataset's own labels, and evidence JSON download

Connected folders

- `src/modules/vision-benchmark/infrastructure/` - `browser-image-processor.ts` and `browser-ollama-provider.ts`, the two adapters that exist so this can work in the tab.
- `src/app/vision-benchmark/` - the page that mounts it.
- `src/modules/vision-benchmark/core/` - `buildVisionPrompt` and the widened schemas.
- `datasets/vision-benchmark/` - the reference dataset this generalises away from.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/vision-benchmarks/`
- `src/app/history/`
- `src/app/api/v1/vision-benchmarks/dataset/`
- `src/components/`
- `src/components/compare/`
- `src/components/history/`
- `src/modules/vision-benchmark/`

src/core/logging/ - the session activity log

Work package: benchmark and provider layer.

What this folder is

A per-session record of what the application did, exportable as JSON, with no account and no user identity attached.

What happened here

New, replacing a generic `logger.ts`. Two rules are built in rather than configurable, and both are there because the obvious design would have been wrong.

Redaction is not optional. The obvious thing for a benchmark log to record is the prompt. The prompt is also the confidential thing this product exists to warn people about - the egress warning tells a user "this text is about to leave your machine", and a log that then writes the same text into a file they can email around has defeated the warning it just issued. So prompt **content** is never recorded by default. Only a length and a short digest, which is enough to prove two runs used the same prompt without disclosing what it was.

Logging is opt-in by header. A request without `x-edgepilot-session` is not logged anonymously and not logged to a default bucket - it is not logged at all. That keeps the default behaviour "we kept no record of what you ran", which is the only default consistent with the rest of the application.

The store is cached on `globalThis` for the same reason the Prisma client is: Next.js hot-reloads modules in development, and a fresh store per reload would drop every session log mid-use.

Files

File	What it is
<code>SessionLog.ts</code>	407 lines. The log, its event and category vocabulary, <code>redact</code> , <code>digestPrompt</code> , the in-memory store, <code>buildSharePayload</code> and <code>SHARE_CONSENT_STATEMENT</code>
<code>sessionLogStore.ts</code>	56 lines. The process-wide store, the <code>x-edgepilot-session</code> header name, and <code>logForRequest</code>

Connected folders

- `src/app/api/v1/session-log/` and `share/` - the endpoints.
- `src/components/dashboard/` - `session.ts` generates the header value; `api.ts` writes the header name out as a literal rather than importing it, because importing this module from the client would instantiate server state at module scope.
- `src/modules/benchmark/core/services/` - `egress-warning.ts`, the warning this log must not undermine.
- `tests/core/` - `session-log.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/components/history/`
- `src/core/`

src/core/quota/ - the cloud-run allowance

Work package: shared.

What this folder is

A cap on how much of the server's cloud quota one visitor may spend.

What happened here

New with hosting. A cloud vision run is 21 requests to Gemini or Groq. With the visitor's own key that is their business; with the key the operator configured, an ungated button is an invitation to spend it all evening. So a visitor gets CLOUD_RUNS_PER_HOUR server-key runs an hour, keyed on their session id, and a message that says when to try again and where to add their own key.

It is in-memory and per process. On a serverless host each instance keeps its own count, so it turns "unlimited" into "a handful" rather than into a hard wall. That is stated in the file rather than implied.

Files

File	What it is
cloudRunQuota.ts	takeCloudRun records a run against the session's bucket if there is room and returns a verdict; resetCloudRunQuota for tests

Connected folders

- src/app/vision-benchmark/ - the server action that calls it.
- tests/core/ - its tests.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/core/

src/modules/benchmark/ - the measurement core

Work package: AI / local model & DevOps - the benchmark and provider layer.

What this folder is

The part of EdgePilot that produces numbers, and the part that decides which numbers it is allowed to produce.

Three layers, and the direction of dependency is the point: `core/` imports nothing from `infrastructure/`. The ports are two interface files. The single place an adapter is constructed is `infrastructure/container.ts`. No API route in this project builds a provider, reads an environment variable, or touches Prisma - which is exactly what makes this module testable with a fake `fetch` and a stepped clock.

What happened here

This is where most of the fix list landed. In summary:

- `hardwareFit` stopped being the constant 50 and became a VRAM residency measurement read from the runtime.
- The first iteration of every run is now a **discarded cold start**. On the same 8B model, time-to-first-token was 36,445 ms cold and 369 ms warm. Averaging that in describes how recently someone used the model, not the model.
- Privacy left the readiness average and became an ordinal class that annotates or disqualifies a recommendation.
- Comparison gained a planner (local models must not run concurrently - they contend for the same GPU) and a report that refuses to declare a winner when the observed ranges overlap.
- Modality and task compatibility became real constraints instead of unread columns.
- A pre-flight egress warning answers "is this prompt about to leave the machine" **before** the request rather than after it.

Hosted-mode pass. The module now has a browser edge: `infrastructure/browser-ollama.ts` runs the unchanged Ollama catalogue, adapter and residency probe from the visitor's tab, and `infrastructure/providers/RecordedProvider.ts` replays what the tab measured through the same runner on the server. `CloudCatalog.ts` lists what a Gemini or Groq key may run; `visitor-keys.ts` lets a visitor's own key stand in for the server's for one request. The hexagonal shape held: the browser reuses `infrastructure`, the application layer gained one interface (`ProviderChain`) and one DTO (`LocalRuntime.ts`), and the core did not change.

Subfolders

Layer	What is in it
<code>core/</code>	Entities, ports, and 13 domain services
<code>application/</code>	DTOs, the runner, and the two use cases
<code>infrastructure/</code>	Provider adapters, Prisma repositories, config, the composition root

Connected folders

- `src/app/api/v1/benchmarks/`, `comparisons/`, `providers/`, `local-runtime/`, `readiness/[id]/` (FOLDER.md) - the HTTP surface.
- `tests/benchmark/` - 19 suites.
- `docs/benchmark/` - the branch guide and the research log.
- `scripts/benchmark/` - the evidence producers.

Referenced from

These folders point here. Each link below resolves in both directions.

- [docs/](#)
- [evidence/benchmark/](#)
- [src/modules/](#)

src/modules/vision-benchmark/ - image classification benchmarks

Work package: vision benchmark.

What this folder is

The second domain module, with the same three layers as the benchmark module and its own copy of the transport helpers - deliberately a copy, so the two modules can change independently.

Its job is narrower and stricter than the text benchmark's: send an image, get back one label from a closed set, and score the result against a manifest whose SHA-256 is recorded so two runs can be compared only when they measured the same thing.

What happened here

Two changes of a completely different size, and the smaller one mattered more.

The one-character fix. `evidence-store.ts` validated filenames against `/^[a-z0-9._-]+\..json$/` - lowercase only - and then built the filename from an ISO timestamp containing an uppercase T and Z. Every live run failed at the write step. The live path had **never worked, once**. One `.toLowerCase()` is the reason this project has any real measurement at all.

The generalisation. The module now accepts arbitrary label sets, so a user can measure a model against their own images. That required widening six fields - TypeScript interfaces and zod schemas together - and building the prompt from the dataset's own labels rather than a fixed seven. All three pre-existing evidence files were re-parsed afterwards to confirm nothing broke.

The warm-up. The first sample was paying the model-load cost. Median latency across the dataset fell from 1322.1 ms to 529.6 ms once a warm-up classification ran first - the same model, the same images.

Hosted-mode pass. A Groq adapter joined the Gemini one in `infrastructure/`, the run request accepts `groq`, and the executor can classify cloud samples in batches. The built-in dataset can now be run from the visitor's browser against their own Ollama (`components/vision/builtInDataset.ts`, served by `api/v1/vision-benchmarks/dataset`), which is what makes the page work hosted.

Files

File	What it is
<code>index.ts</code>	17 lines. The module's public surface

Subfolders

Layer	What is in it
<code>core/</code>	Types, schemas, metrics, normalisation, the workload prompt
<code>application/</code>	Executor, evaluator, dashboard projection, the provider port
<code>infrastructure/</code>	Server and browser adapters, image processing, manifest loading, evidence writing

Connected folders

- `src/app/vision-benchmark/` - the page.
- `src/app/api/v1/vision-benchmarks/` - the server-side endpoint.
- `src/components/vision/` - the browser path.

- [datasets/vision-benchmark/](#), [evidence/vision-benchmark/](#), [scripts/vision-benchmark/](#), [docs/vision-benchmark/](#), [tests/vision-benchmark/](#).

Referenced from

These folders point here. Each link below resolves in both directions.

- [datasets/](#)
- [docs/](#)
- [src/modules/](#)

src/shared/types/ - the scaffold contract

Work package: integration and API surface.

What this folder is

Four small files pairing a TypeScript type with the zod schema that validates it, so validation and typing cannot disagree.

What happened here

`device.ts` was deleted with the device module, and `index.ts` lost its export of it - a one-line change that is the visible end of a removal touching the UI, the API, the module and the database.

`benchmark.ts` changed by three lines. That small number is deliberate: the benchmark module's own DTO is a **superset** of this shape rather than a replacement, keeping every field it already had, in the same place, with the same name, so the dashboard being written against this contract did not have to be rewritten.

Files

File	What it is
<code>benchmark.ts</code>	46 lines. <code>BenchmarkRequest</code> , <code>BenchmarkResult</code> , <code>ReadinessScore</code> and their schemas
<code>provider.ts</code>	46 lines. <code>Provider</code> , <code>AIProvider</code> , <code>AIResponse</code> , <code>ProviderBenchmarkResult</code>
<code>workload.ts</code>	17 lines. <code>Workload</code> , <code>CreateWorkloadRequest</code> and their schemas
<code>index.ts</code>	4 lines. Re-exports. Three lines now, where there were four

Connected folders

- `src/modules/benchmark/application/dtos/` - the superset envelope.
- `src/app/api/v1/workloads/` - validates against `CreateWorkloadSchema`.
- `src/shared/` - the parent.

src/app/api/v1/ - the versioned API

Work package: integration and API surface, with individual routes belonging to their modules.

What this folder is

Eleven endpoints. Every one of them follows the same envelope - `{ success, data }` on the way out, `{ success: false, error, details }` on failure - because the dashboard renders exactly one error shape and any deviation costs a special case in the UI.

None of these files constructs an adapter, reads an environment variable, or touches Prisma directly. They ask `src/modules/benchmark/infrastructure/container.ts` for a wired use case and get one. That is what keeps a route to parse, dispatch and choose a status code.

What happened here

Four endpoints are new - `comparisons`, `local-runtime`, `session-log` and `session-log/share`. One was removed - `devices`. And one, `workloads`, was fixed from a scaffold that never called the database at all.

Ownership across all of them moved from a single hardcoded shared user to a per-session user derived from the `x-edgepilot-session` header. Under the old behaviour a hosted deployment would have handed every visitor every other visitor's rows.

Subfolders

Endpoint	What it does
<code>benchmarks/</code>	POST run and record; GET list for one owner
<code>comparisons/</code>	POST compare two or more models
<code>local-runtime/</code>	GET is Ollama running, and what does it have
<code>providers/models/</code>	GET <code>?provider=gemini groq</code> — which models the server's key for that vendor may run. The cloud counterpart of <code>local-runtime</code>
<code>providers/</code>	GET the provider catalogue and what is configured
<code>readiness/</code>	GET the readiness score for one benchmark
<code>session-log/</code>	GET export, DELETE discard
<code>vision-benchmarks/</code>	GET/POST the vision workload
<code>workloads/</code>	POST create a workload row; GET list

Connected folders

- `src/modules/benchmark/infrastructure/` - the composition root every route calls.
- `src/lib/` - `sessionOwner.ts`, how these routes decide who owns a row.
- `docs/internal/` - the wire contract.
- `tests/benchmark/` - `benchmarks-route.test.ts` drives the main route with the container mocked.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/`
- `src/components/dashboard/`

- `src/modules/`
- `src/shared/`

src/modules/benchmark/application/ - use cases and envelopes

Work package: benchmark and provider layer.

What this folder is

The layer between HTTP and the domain. It orchestrates: validate, measure, persist, return.

What happened here

The runner grew by 269 differing lines, carrying the cold-start iteration and the residency probe. The use case shrank slightly while gaining an ownership rule that had been implicit.

The design decision that survived every change: **persistence is layered on top of measurement, not inside it.** BenchmarkRunner knows nothing about the database, so when Postgres is unreachable mid-run the measurement is still returned to the caller with persisted: false, rather than being thrown away. A benchmark that ran is evidence whether or not it was saved.

Subfolders

Subfolder	What is in it
dtos/	The typed measurement envelope and the request schema
services/	BenchmarkRunner
use-cases/	RunBenchmark, RunComparison

Connected folders

- ../core/ - the domain services it calls.
- ../infrastructure/ - the adapters it is handed.
- tests/benchmark/.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/benchmark/

src/modules/benchmark/core/ - the domain

Work package: benchmark and provider layer.

What this folder is

Pure domain logic. Nothing in here imports from `infrastructure/`, nothing here performs I/O, and nothing here knows what HTTP is. That constraint is what lets the whole scoring layer be tested without a network, a model, or a database.

What happened here

Eight new service files, 1,761 lines between them, and every one of them replaced something that was previously a constant, an assumption, or nothing at all. The single deletion that matters as much as the additions: `ReadinessCalculator` lost its privacy term.

Subfolders

Subfolder	What is in it
<code>entities/</code>	Benchmark, BenchmarkResult, ReadinessScore
<code>ports/</code>	The two interfaces the infrastructure layer implements
<code>services/</code>	10 files - scoring, classification, comparison, privacy

Connected folders

- `../application/` - what calls these.
- `../infrastructure/` - what implements the ports.
- `tests/benchmark/`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/`

src/modules/benchmark/infrastructure/ - adapters and wiring

Work package: benchmark and provider layer.

What this folder is

Everything that talks to something outside the process - HTTP, Prisma, the environment - plus the one place where it is all wired together.

What happened here

Two new adapters, and both of them exist to answer a question the application used to answer too late or not at all. `OllamaResidencyProbe.ts` reads what actually happened to memory during a local run, which is what turned hardware fit from a constant into a measurement. `OllamaCatalog.ts` answers "is Ollama running and what does this machine have" **before** a benchmark is attempted rather than after one fails. `config.ts` grew by 41 lines and `container.ts` by 37, both because there is more to wire.

Files

File	What it is
<code>container.ts</code>	78 lines. The composition root. Routes ask for a fully wired use case and get one; they never construct an adapter, read an environment variable, or touch Prisma. Every wiring decision is in this one readable place
<code>config.ts</code>	184 lines. Reads and validates the environment once, with documented defaults, so a misconfiguration surfaces as a named variable rather than a stack trace three layers down. Also refuses to evaluate in a browser bundle - every value here is a secret or a host that must stay server-side. The guard is a runtime check because <code>server-only</code> is not a dependency, and it is still worth having: it turns an accidental client import into an immediate, explicit failure at the point of the mistake
<code>CloudCatalog.ts</code>	The same question for Gemini and Groq: which models may the configured key run, asked over each vendor's list-models endpoint with the key in a header. Narrowed to text generators; audio, embedding and safety models are counted, not shown. Replaces the free-text model box and its hardcoded suggestion list
<code>OllamaCatalog.ts</code>	223 lines. Runtime status and the installed-model list. Exists because <code>isConfigured()</code> only reports whether a host string is set - whether anything is listening is a runtime question, and the user was learning the answer from a failed benchmark
<code>visitor-keys.ts</code>	A visitor's own Gemini/Groq keys, read from two request headers, laid over the server's config for that one request. A registry built with them is never cached. Never logged, stored or echoed
<code>browser-ollama.ts</code>	The visitor's OWN Ollama, reached from their browser tab. Hosted, the server's "localhost" is a datacentre container with no Ollama; only the page in the visitor's browser can reach theirs. Reuses <code>OllamaCatalog</code> , <code>OllamaProvider</code> and <code>OllamaResidencyProbe</code> unchanged (they are plain fetch), adds the CORS remedy (<code>OLLAMA_ORIGINS</code>) a hosted page needs, and packages a run as the recorded field the benchmark and comparison routes accept. Never reads <code>process.env</code>
<code>OllamaResidencyProbe.ts</code>	107 lines. <code>GET /api/tags</code> for size on disk, <code>GET /api/ps</code> for size (total resident, including the KV cache for the context window) and <code>size_vram</code> (how many of those bytes are on the GPU). <code>size_vram / size</code> is the fit signal, and it is reported by the runtime rather than inferred by us

Subfolders

Subfolder	What is in it
<code>providers/</code>	The five adapters, the registry, the error vocabulary, transport helpers
<code>repositories/</code>	The Prisma implementations

Connected folders

- `../core/ports/` - the interfaces implemented here.
- `../core/services/` - `HardwareFitAssessor` consumes the probe's observation.
- `src/app/api/v1/` - every route calls `container.ts` and nothing else here.
- `src/app/api/v1/local-runtime/` - the catalogue's endpoint.
- `tests/benchmark/` - `benchmark-config.test.ts`, `ollama-catalog.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `prisma/`
- `src/app/api/v1/benchmarks/`
- `src/app/api/v1/providers/models/`
- `src/app/setup/`
- `src/modules/benchmark/`
- `src/modules/benchmark/application/`
- `src/modules/benchmark/application/services/`
- `src/modules/benchmark/core/`

src/modules/vision-benchmark/application/ - running and grading

Work package: vision benchmark.

What this folder is

Four files: the executor that runs a workload sample by sample, the evaluator that grades the result against thresholds, the dashboard projection, and the provider port.

What happened here

executor.ts grew from 159 to 222 lines, all of it for the warm-up classification - and getting that right took three corrections worth recording, because each one is a different class of mistake:

- It **consumed the deterministic test clock**, moving every timing in the suite. Fixed by calling `provider.classify()` directly rather than the `timedClassifySample` wrapper.
- It **fired on cloud providers**, which meant a billable API call and one extra image sent to a third party on every run. Fixed: local providers only, plus an explicit `warmUp?: boolean` so a caller can always opt out.
- It **ate the first scripted response** in two existing tests. Those two now opt out with a comment explaining why, and two new tests cover the warm-up properly.

The warm-up's failure is swallowed on purpose: if the throwaway classification errors, the measured run should still proceed.

The executor also threads `labels`, `datasetId` and `workloadId` through, which is what lets a run be about a dataset other than the built-in one.

Hosted-mode pass. `executeVisionBenchmark` gained a concurrency option, default 1. Batches of that size are awaited in turn and each response is written to its sample's own slot, so a batch of one is the old sequential loop byte for byte. Cloud runs from the page use 7 to fit a serverless time limit; a local run stays at 1, because two requests to one GPU measure contention rather than the model.

Files

File	What it is
executor.ts	223 lines. <code>executeVisionBenchmark</code> , the warm-up, the injected clock
evaluator.ts	162 lines. <code>evaluateVisionBenchmark</code> and <code>DEFAULT_VISION_THRESHOLDS</code> - 0.80 accuracy, 0.75 macro-F1, 0.05 maximum invalid-output rate, 0.95 successful-request rate
dashboard.ts	50 lines. <code>toVisionDashboardRow</code> , <code>rankVisionDashboardRows</code>
provider.ts	29 lines. <code>VisionProvider</code> and <code>VisionImageProcessor</code> - the two ports the infrastructure layer implements

Connected folders

- `../core/` - the types, metrics and thresholds.
- `../infrastructure/` - the four provider and two processor implementations of these ports.
- `tests/vision-benchmark/` - `vision-execution.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/vision-benchmark/`

src/modules/vision-benchmark/core/ - the vision domain

Work package: vision benchmark.

What this folder is

Five files defining what a vision benchmark **is**: the sample and evidence shapes, the zod schemas that validate them, the metric calculations, the label normaliser, and the prompt.

What happened here

This is where the generalisation to arbitrary datasets was actually done, and where it went wrong first.

schemas.ts had perClass pinned to .length(VISION_LABELS.length) - exactly seven. A dataset with a different class count ran to completion and *then* failed validation, which is the worst possible ordering: all the cost, none of the result. Widening it needed six fields changed across types.ts and schemas.ts **together**; changing one without the other cost three separate next build round-trips, because Jest type-checks only what it imports.

workload.ts grew from 10 lines to 31 for buildVisionPrompt(labels). A deliberate failure run proved it was necessary: with the fixed seven-label prompt and a custom two-class dataset, the result was 0% accuracy and 100% invalid output. Re-run with a prompt built from the dataset's own labels: 100%, passed.

normalization.ts is the security-relevant file here. EdgePilot never *acts* on model output, so the threat is not execution - it is output being read as a **verdict**. The defence is exact matching against a closed label set, with anything else recorded as invalid_output rather than guessed at.

Hosted-mode pass. VisionBenchmarkRunRequestSchema accepts provider: 'groq' alongside ollama and gemini.

Files

File	What it is
types.ts	202 lines. VisionBenchmarkSample, VisionBenchmarkEvidence, ClassMetrics, VisionDashboardRow, VISION_LABELS, VISION_DATASET_ID, VISION_WORKLOAD_ID
schemas.ts	278 lines. The zod counterparts, including the manifest and evidence schemas
metrics.ts	171 lines. Exact-match accuracy, macro precision/recall/F1, invalid-output rate, successful-request rate, median and p95 latency, throughput, and the per-class matrix
normalization.ts	60 lines. normalizeVisionLabel - exact matching against the closed set
workload.ts	32 lines. buildVisionPrompt(labels), VISION_BENCHMARK_PROMPT, and the prompt and workload version constants that make two runs comparable

Connected folders

- ../application/ - the executor and evaluator.
- ../infrastructure/ - what produces the evidence these schemas validate.
- src/components/vision/ - the upload that calls buildVisionPrompt with a user's own labels.
- evidence/evaluation/ - the injection cases drive normalizeVisionLabel directly.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/evaluation/
- src/modules/vision-benchmark/

src/modules/vision-benchmark/infrastructure/ - **adapters**

Work package: vision benchmark.

What this folder is

Nine files. Two providers and one image processor for the server; one provider and one image processor for the **browser**; plus manifest loading, evidence writing, transport and the run service.

What happened here

The bug that mattered most in the whole project is in this folder. `evidence-store.ts` validated filenames against a lowercase-only pattern and then built them from an ISO timestamp containing T and Z. Nine lines differ; one of them is a `.toLowerCase()`; and before it, no live run had ever managed to write its evidence.

The two browser adapters are new, and they exist for a reason worth stating plainly.

`SharpVisionImageProcessor` reads files from disk with a Node library, and `OllamaVisionProvider` builds its base64 with `Buffer`, which does not exist in a browser. For a user's own photographs both are the wrong machine twice over: the images would have to travel to the server, and the model measured would be the server's rather than the user's. So the browser pair prepares images in the tab and talks straight to the user's own Ollama on localhost.

Two details in them are load-bearing:

- **The version string is deliberately different.** `browser-canvas-png-v1`, not `sharp-png-v1`. Canvas and sharp do not resample or encode PNG identically, so runs prepared by each must never be treated as comparable by accident.
- **Determinism is preserved.** `temperature: 0` and `seed: 42`, identical to the server adapter.

Both browser files also hit the same TypeScript trap twice: with `target: es5` and no `downlevelIteration`, spreading a typed array is a hard compile error. Both now use `String.fromCharCode.apply(null, Array.from(...))`.

Files

File	What it is
<code>evidence-store.ts</code>	106 lines. <code>FileVisionEvidenceStore</code> - writes the artefacts. The filename fix lives here
<code>browser-image-processor.ts</code>	181 lines. Canvas resize to 512, <code>crypto.subtle</code> SHA-256, JPEG APP1 EXIF detection. <code>preprocessingVersion = 'browser-canvas-png-v1'</code>
<code>browser-ollama-provider.ts</code>	171 lines. <code>providerName = 'ollama-browser'</code> , posts to the user's own <code>/api/chat</code> from the tab
<code>image-processor.ts</code>	174 lines. <code>SharpVisionImageProcessor</code> - the server-side equivalent
<code>ollama-provider.ts</code>	139 lines. The server-side local vision adapter
<code>gemini-provider.ts</code>	174 lines. The Gemini cloud vision adapter
<code>groq-provider.ts</code>	The Groq cloud vision adapter: OpenAI chat-completions dialect, image as a data URL, <code>temperature 0</code> , key in the Authorization header. Only the Llama 4 family accepts images; the catalogue marks which
<code>manifest-loader.ts</code>	50 lines. Loads and validates <code>datasets/vision-benchmark/manifest.json</code> , including its SHA-256
<code>run-service.ts</code>	80 lines. <code>runVisionBenchmarkRequest</code> - what the API route calls

File	What it is
<code>http.ts</code>	38 lines. The injected <code>fetch</code> and <code>clock</code> . A deliberate copy of the benchmark module's, not a shared import

Connected folders

- `../application/` - the ports these implement.
- `src/components/vision/` - the upload panel that mounts the two browser adapters.
- `evidence/vision-benchmark/` - what `evidence-store.ts` writes, and why one file there is explicitly un-ignored.
- `datasets/vision-benchmark/` - what `manifest-loader.ts` reads.
- `src/app/api/v1/vision-benchmarks/` - the route that calls `run-service.ts`, and why it sets runtime explicitly.
- `tests/vision-benchmark/` - `vision-infrastructure.test.ts`, `vision-providers.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/vision-benchmark/`
- `src/modules/vision-benchmark/core/`

src/app/api/v1/benchmarks/ - run and record a benchmark

Work package: benchmark and provider layer.

What this folder is

POST /api/v1/benchmarks runs a benchmark and records it. GET lists recorded benchmarks for one owner.

What happened here

The response envelope is unchanged from the scaffold, deliberately - the dashboard is written against it, and everything new was added *inside* data rather than beside it.

This route is server-side only and must stay that way: it is the only thing in the request path that can see GEMINI_API_KEY and GROQ_API_KEY.

The measurement it now returns is a different thing from what it used to return: a real hardware fit instead of the constant 50, a discarded cold-start iteration reported separately, and a readiness score with privacy taken out of the average.

Hosted-mode pass. POST accepts an optional recorded field: a run the visitor's browser already measured against its own Ollama (iterations + 1 responses, the first a cold start, plus residency readings before and after). With it, the route scores through RecordedProvider instead of calling a provider; without it, nothing changed. Two optional headers carry a visitor's own Gemini and Groq keys for that request only. The activity log records measured_by: browser | server and the local host, never a key.

Files

File	What it is
route.ts	174 lines. POST and GET, plus dynamic and maxDuration. maxDuration matters - a cold local model can take over half a minute to answer its first request

Connected folders

- src/modules/benchmark/application/use-cases/ - RunBenchmark, what this dispatches to.
- src/modules/benchmark/infrastructure/ - container.ts, where the wired use case comes from.
- tests/benchmark/ - benchmarks-route.test.ts.
- docs/internal/ - benchmark-api.md.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/modules/benchmark/

src/app/api/v1/comparisons/ - **compare two or more models**

Work package: benchmark and provider layer.

What this folder is

POST /api/v1/comparisons. New in this version.

What happened here

The response is deliberately three separate things rather than one verdict:

- **plan** - what was decided *before* anything ran: whether the entrants are comparable at all, whether they had to run one after another, and why. A reader who disagrees with the conclusion can check the method first.
- **outcomes** - each run in full and unchanged, so nothing is available only in summarised form.
- **report** - per-dimension winners, each marked established or not.

Nothing is written to the database. A comparison analyses runs; it is not a new kind of record, and making it depend on a workload row would tie a read-only question to the write path.

There is **no UI for this endpoint yet**. It works and is tested; nothing in the dashboard calls it.

Hosted-mode pass. Each entrant may carry a recorded measurement from the visitor's browser (Ollama only; length must be `iterations + 1`, enforced in `superRefine`). Families and parameter size for a local entrant now arrive from the browser's own catalogue too, since the server's `localModelCatalogue()` sees nothing when hosted. `parseParameterSize` moved to `application/dtos/LocalRuntime.ts` so both sides use one parser. The visitor's own cloud keys apply per request.

Files

File	What it is
<code>route.ts</code>	290 lines. POST, returning plan, outcomes and report

Connected folders

- `src/modules/benchmark/application/use-cases/` - `RunComparison`.
- `src/modules/benchmark/core/services/` - `ComparisonPlanner` and `ComparisonReport`, the two files that make the verdict defensible.
- `tests/benchmark/` - `comparison.test.ts`, `run-comparison.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/app/compare/`
- `src/components/compare/`
- `src/modules/benchmark/`

src/app/api/v1/local-runtime/ - is Ollama running

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/local-runtime. New in this version.

What happened here

It answers a question the application previously answered too late. `OllamaProvider.isConfigured()` reports only whether a host string is set - by design, because whether anything is *listening* is a runtime question. The consequence was that a user with Ollama stopped, or on the wrong port, found out from a failed benchmark rather than from the screen where they picked the provider.

Two fields carry the design. `ok` is the single value a caller needs to decide whether to enable a Run button. `remedy` is what to show when `ok` is false - never a bare failure, always the command that fixes it.

No credential is involved and no prompt is sent: this reads a version string and a model list.

Hosted-mode pass. This route describes the **server's** machine, which hosted is a container with no Ollama. The dashboard, the comparison and the vision page no longer call it; they probe the visitor's own Ollama from the browser (`infrastructure/browser-ollama.ts`) and get the same DTO, now shared through `application/dtos/LocalRuntime.ts`. The route stays for the CLI and for a server that sits beside an Ollama.

Files

File	What it is
<code>route.ts</code>	73 lines. GET, returning runtime state, installed models, and a remedy when something is wrong

Connected folders

- `src/modules/benchmark/infrastructure/` - `OllamaCatalog.ts`, which does the work.
- `src/components/dashboard/` - `InstalledModels.tsx`, which renders it.
- `tests/benchmark/` - `ollama-catalog.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/modules/benchmark/`

src/app/api/v1/providers/ - the provider catalogue

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/providers - which providers exist, and whether each is usable on this server right now.

GET /api/v1/providers/models?provider=gemini|groq (in models/) - which models the server's key for that vendor may run. The dashboard's model field for a cloud provider is a list drawn from this, the way the Ollama field is drawn from local-runtime. The query is validated with Zod; the key is read from server configuration and never appears in the response.

What happened here

The behaviour worth knowing is what it does when the database is **not** reachable. It still answers, from the registry alone, with provider_id: null and a message explaining why. The dashboard can still render the list and say which providers are configured. Degrading to a useful answer beats returning a 500 for a question that does not strictly need the database.

No credential, and no fragment of one, appears in the response.

Files

File	What it is
route.ts	92 lines. GET, returning the catalogue with is_configured, configuration_hint, database availability, and server configuration warnings

Connected folders

- src/modules/benchmark/infrastructure/providers/ - ProviderRegistry.ts.
- prisma/ - seed.ts is what puts the catalogue rows there.
- src/components/dashboard/ - ProviderPanel.tsx.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/modules/benchmark/
- models/

`src/app/api/v1/readiness/`

`src/app/api/v1/readiness/` - readiness scores

Work package: benchmark and provider layer.

What this folder is

A container for the dynamic segment below it.

Subfolders

- `[id]/` (FOLDER.md) - GET `/api/v1/readiness/[id]`.

Connected folders

- `src/app/api/v1/` - the rest of the API.
- `src/modules/benchmark/core/services/` - `ReadinessCalculator.ts`, which produces what this reads back.

src/app/api/v1/session-log/ - export or discard the session record

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/session-log exports this session's activity as JSON. DELETE discards it. New in this version.

What happened here

The session is identified by the x-edgepilot-session header, which the client generates. There is no account, no cookie and no server-assigned identity: a caller that stops sending the header simply has no log.

GET returns a **downloadable document** rather than an API envelope, because the point of it is to be saved and attached to something. The disclosure block travels inside the file, so a copy that changes hands still states what was and was not recorded - which matters, because prompt content is never in it.

There is **no UI for this endpoint yet**.

Files

File	What it is
route.ts	85 lines. GET export, DELETE discard

Subfolders

- share/ - the consent-gated upload.

Connected folders

- src/core/logging/ - SessionLog.ts and the store.
- src/components/dashboard/ - session.ts, which generates the header value.
- tests/core/ - session-log.test.ts.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/app/history/
- src/core/

src/app/api/v1/vision-benchmarks/ - the vision workload endpoint

Work package: vision benchmark.

What this folder is

GET and POST /api/v1/vision-benchmarks - run the image-classification workload server-side, and list what has been run.

What happened here

The route sets `runtime` explicitly, because the server-side image processor uses `sharp`, a native module that cannot run on the edge runtime.

Its role narrowed rather than grew. It is now the *server* path for the vision workload - used for the controlled dataset. When a user brings their own images, nothing goes through this endpoint at all: the browser prepares the images and talks straight to the user's own Ollama, so the photographs never leave the machine.

Files

File	What it is
route.ts	128 lines. GET and POST, with <code>dynamic</code> and <code>runtime</code> set
dataset/	GET the built-in dataset described for a browser about to run it (samples, labels, prompt, digest); dataset/images/<file> serves one fixture, matched against the manifest's own list

Connected folders

- src/modules/vision-benchmark/ - the module.
- src/modules/vision-benchmark/infrastructure/ - `run-service.ts` and the `sharp` processor.
- src/components/vision/ - the browser path that deliberately bypasses this route.
- tests/vision-benchmark/ - `vision-api.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- dataset/

src/app/api/v1/workloads/ - workload rows

Work package: integration and API surface.

What this folder is

POST /api/v1/workloads creates a workload row and returns its real uuid. GET lists the workloads belonging to the caller.

What happened here

This is the single most consequential fix in the project. The committed version was a scaffold: it returned the literal string `temp-workload-id` and never called Prisma. So nothing the dashboard saved ever reached the database, and every benchmark run afterwards failed with `404 workload not found`. The dashboard had grown a workaround for it - detect the placeholder id, warn the user the row was not saved, and offer a field to paste a real database uuid into by hand.

The route went from 53 lines to 126 and now writes real rows. Ownership comes from the caller's session id, so a row belongs to whoever created it without anyone signing up - and without a uuid ever being shown to, or typed by, a user. The dashboard's paste-a-uuid fallback and the field that went with it are gone.

The original scaffold is kept beside the route as `route.ts.scaffold.bak` and is referenced **by name** in the new file's header, which is why it is explicitly un-ignored in `.gitignore` while every other `.bak` is ignored.

Files

File	What it is
<code>route.ts</code>	126 lines. POST and GET, both scoped to the caller's session
<code>route.ts.scaffold.bak</code>	The original 53-line stub, kept as the reference the docblock points at

Connected folders

- `src/lib/` - `sessionOwner.ts`, where the owner comes from.
- `src/components/dashboard/` - `SetupPanel.tsx`, which lost 379 lines' worth of workaround because of this.
- `scripts/dev/` - `seed-demo-rows.ts`, which existed because this endpoint did not persist.
- `prisma/` - the workloads table.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/shared/types/`

src/modules/benchmark/application/dtos/ - what goes in, what comes out

Work package: benchmark and provider layer.

What this folder is

Two files. `BenchmarkRequest.ts` validates what comes **in**. `BenchmarkMeasurement.ts` validates what goes **out**, and is the reason a reader can tell a measurement from a derivation from a placeholder.

What happened here

`BenchmarkMeasurement.ts` grew by 38 differing lines to carry the cold start as a first-class object rather than a footnote: `ColdStart` and `ColdStartSchema` exist so the discarded iteration has a name, a duration and a row of its own.

It is deliberately a **superset** of the scaffold's `BenchmarkResponseSchema` rather than a replacement. The dashboard was already being written against the existing response shape, so every field it had is still there, in the same place, with the same name, and everything new was added inside. That is why `src/shared/types/benchmark.ts` needed only three lines changed.

Files

File	What it is
<code>BenchmarkMeasurement.ts</code>	227 lines. <code>MeasuredIteration</code> , <code>MeasurementSummary</code> , <code>ColdStart</code> , <code>FallbackAttempt</code> , <code>BenchmarkRun</code> , <code>MeasurementStatus</code> , <code>ProviderErrorCode</code> , and <code>summarise()</code> - the function where a number would be invented if anywhere
<code>BenchmarkRequest.ts</code>	<code>BenchmarkRequestSchema</code> , plus <code>RecordedMeasurementSchema</code> - a run the visitor's browser measured against its own Ollama (<code>iterations + 1</code> responses, the first a cold start, and the residency read before and after). Accepted only with <code>provider: ollama</code>
<code>LocalRuntime.ts</code>	The local-runtime DTO and <code>toLocalRuntimeDto</code> , shared by <code>GET /api/v1/local-runtime</code> and the browser probe so the two answers are interchangeable; <code>parseParameterSize</code> ("7.2B" to billions) lives here for the same reason

Connected folders

- `../services/` - `BenchmarkRunner` produces these.
- `src/components/dashboard/` - `api.ts` imports these types with `import type`, so the dashboard cannot drift from the envelope without a compile error.
- `tests/benchmark/` - `benchmark-measurement.test.ts` is about what `summarise()` **refuses** to report.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/application/`
- `src/shared/`
- `src/shared/types/`

src/modules/benchmark/application/services/ - the runner

Work package: benchmark and provider layer.

What this folder is

One file, and the most-changed file in the module: 662 lines, 269 of them differing from the team's version.

It runs one benchmark request against the provider chain and turns raw adapter output into the typed envelope. It does no I/O of its own beyond the providers it is handed, and it knows nothing about the database.

What happened here

The cold start. The runner now runs N+1 iterations and discards the first. Measured on the same 8B model in one session: TTFT 36,445 ms cold against 369 ms warm, a factor of 99, with readiness swinging 75 to 90 on load state alone. The discarded iteration is not deleted - it is reported with its own duration and stored with `warmup: true`, so the arithmetic can be checked from the database rows.

The counting bug that followed. With the warm-up added, a request for three iterations reported "4/4 iterations succeeded". Success, dominant failure code and the summary now all read from `responses.slice(1)`. The fix went into the runner rather than into the assertion that caught it.

The residency probe. A pre-run probe records what memory looked like before the model was asked anything, which is what hardware fit is computed against. Its result is null-guarded explicitly - the first version's catch produced the right value for the wrong reason, which would have masked any other fault.

The fallback policy, unchanged and worth restating: fall back only when the provider failed for a reason another provider could plausibly not share.

Hosted-mode pass. The runner's constructor now takes a `ProviderChain` - has and `chainFor` - rather than the concrete registry. The registry satisfies it; so does the single-provider chain a browser-recorded run is scored through (`RecordedProvider`), which is how a run measured in the visitor's browser goes through exactly this code and no other.

Files

File	What it is
BenchmarkRunner.ts	663 lines. The run loop, the cold start, the residency probe, the fallback chain, and <code>DEFAULT_UNMEASURED_INPUTS</code>

Connected folders

- `../dtos/` - the envelope it produces.
- `../use-cases/` - `RunBenchmark`, which persists what it returns.
- `../../core/services/` - hardware fit, readiness, task compatibility.
- `../../infrastructure/` - `OllamaResidencyProbe.ts`.
- `prisma/migrations/` - the warmup column.
- `tests/benchmark/` - `benchmark-runner.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/application/`

src/modules/benchmark/application/use-cases/ - orchestration

Work package: benchmark and provider layer.

What this folder is

Two use cases: run a benchmark and persist it, and compare several runs without persisting anything.

What happened here

`RunComparison.ts` is new. `RunBenchmark.ts` changed by 90 lines, and two of its decisions are worth reading before anyone changes them - both are written into the file's own header.

Where `userId` comes from. Ownership is derived from the workload the request names: `workloads.userId` is required, and the benchmark is about that workload. This began as an interim rule while there was no session. It survived now that there *is* one, because it is still the right answer - a benchmark belongs to whoever owns the thing being benchmarked, and the workload row is already owned by the session that created it.

Why `RunComparison` does not persist. A comparison is an analysis of runs, not a new kind of record. Making it depend on a workload row would tie a read-only question to the write path. The individual outcomes come back in full, so a caller that wants to store them can.

`RunComparison` also honours the plan's execution mode rather than always reaching for `Promise.all` - which matters, because running two local models concurrently measures GPU contention rather than the models.

Hosted-mode pass. `RunComparison` takes a second factory, `createRecordedRunner`, used for any entrant that arrives with a recorded measurement from the visitor's browser. `RunBenchmark` is unchanged: the container hands it a runner built around the recording, and it validates, persists and returns as it always did.

Files

File	What it is
<code>RunBenchmark.ts</code>	309 lines. Validate the referenced rows, run, persist, return. Includes <code>statusForFailedRun</code> and the graceful-degradation path when the database is down mid-run
<code>RunComparison.ts</code>	213 lines. Decide comparability, decide execution mode, run, report per-dimension verdicts

Connected folders

- `../services/` - the runner both call.
- `../../core/services/` - `ComparisonPlanner` and `ComparisonReport`.
- `../../infrastructure/repositories/` - persistence and the context lookup.
- `src/app/api/v1/benchmarks/` and `comparisons/` - the routes.
- `tests/benchmark/` - `run-benchmark.test.ts`, `run-comparison.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/application/`

`src/modules/benchmark/core/entities/`

`src/modules/benchmark/core/entities/` - domain entities

Work package: benchmark and provider layer.

What this folder is

One file holding the three shapes the domain reasons about.

What happened here

`Benchmark.ts` changed by 14 lines: `deviceId` came off `Benchmark`, and `ReadinessScore` gained a nullable `hardwareFit` and a `privacyClass`. Those two type changes are the domain-level statement of two decisions - that a hardware fit may be genuinely unknown rather than defaulted, and that privacy is a class and not a score.

Files

File	What it is
<code>Benchmark.ts</code>	49 lines. <code>Benchmark</code> , <code>BenchmarkResult</code> , <code>ReadinessScore</code>

Connected folders

- `../services/` - what computes these.
- `../../infrastructure/repositories/` - what maps them to and from Postgres.
- `prisma/` - the tables they correspond to.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/core/`

src/modules/benchmark/core/ports/ - the boundary

Work package: benchmark and provider layer.

What this folder is

Two interface files, 44 lines together. They are the whole reason this module can be called hexagonal without qualification: the domain declares what it needs, and infrastructure/ supplies it.

Because these are interfaces and nothing else, the test suite can hand the application layer a FakeProvider and an in-memory repository and exercise the real runner and the real use case with no network and no database.

What happened here

Unchanged. That is the interesting part - eight new services, a new use case, a removed module and four migrations all landed without needing to move the boundary. A boundary that survives that much change is one that was drawn in the right place.

Files

File	What it is
AIPProvider.ts	26 lines. What a provider must be able to do
BenchmarkRepository.ts	18 lines. What persistence must be able to do

Connected folders

- ../../infrastructure/providers/ - five implementations of AIPProvider.
- ../../infrastructure/repositories/ - the Prisma implementation of BenchmarkRepository.
- tests/benchmark/ - helpers.ts implements both with test doubles.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/benchmark/core/
- src/modules/benchmark/infrastructure/

src/modules/benchmark/core/services/ - **scoring and judgement**

Work package: benchmark and provider layer.

What this folder is

Ten files, and collectively they are the answer to "what is this application allowed to claim". Eight of them are new.

What happened here

Each file below records, in its own header, the thing it was written to stop. The short version:

- A **constant** was being presented as a hardware measurement.
- A **weighted average** was mixing legal exposure with throughput.
- A **mean** was being reported as a comparison result.
- A **declared task type** was being written to the database and read by nothing.
- **Two local models** were about to be benchmarked concurrently on one GPU.
- A **privacy verdict** was arriving after the prompt had already left.

Files

File	What it is
HardwareFitAssessor.ts	229 lines. Replaces hardwareFit: 50, a constant every device on earth received. Scores placement - did the model fit in GPU memory, and if not how much of it did - and deliberately not speed, because latency is already a readiness term and a slow CPU run would otherwise be punished twice. Reports VRAM taken and the spilled share in words, and says so when the runtime does not report card capacity
ReadinessCalculator.ts	89 lines. Four measured performance terms. Privacy was the fifth and is gone: a coarse label mapped to 30/60/100 let a fast provider average away a data policy that should have disqualified it, and implied an arithmetic relationship between privacy levels nobody can defend
PrivacyAssessor.ts	314 lines. Privacy as an ordinal class , with disqualifiers, egress classification and observed transport. Annotates or rules out a recommendation instead of nudging a number. Fails closed on a null endpoint
privacy-catalogue.ts	97 lines. Provider data-policy facts by slug and billing tier. Every value starts null and unverified on purpose - these are terms-of-service facts that differ by tier and change without notice, and filling them in from memory is how an evidence-based tool starts publishing stale claims confidently
egress-warning.ts	133 lines. Answers <i>before</i> the run: is this prompt about to leave the machine, and to whom. Pure and synchronous so any surface can use it. Reports the prompt's character count and never carries the prompt text - an object that warns text is about to leave must not contain a copy of it
ComparisonPlanner.ts	141 lines. Decides whether entrants may be compared at all, and whether they may run concurrently. For two cloud providers, parallel is right. For two local models it destroys the measurement: they contend for the same GPU and the runtime may evict one for the other
ComparisonReport.ts	506 lines. Per-dimension verdicts, each marked established or not. A mean is not a fact: if the entrants' observed ranges overlap, no winner is declared. At five iterations a 10% gap in means routinely sits inside a 3x spread
ModelModality.ts	167 lines. Vision and embedding are identified positively from Ollama's families array; text is the residual and is always marked inferred , because the absence of a vision family is not proof of anything
TaskCompatibility.ts	174 lines. Turns a declared task type into a constraint. Deliberately asymmetric - vision can do text, text cannot do vision, embedding neither. shapeOf('code_generation') states plainly that nothing here compiles or runs the output
ComparisonEngine.ts	101 lines. The scaffold's original comparison helper

Connected folders

- `../../application/services/` - the runner that calls these.
- `../../application/use-cases/` - `RunComparison`, which uses the planner and the report.
- `../../infrastructure/` - `OllamaResidencyProbe.ts` supplies the observation hardware fit scores.
- `src/core/logging/` - the session log, which must not undo the egress warning.
- `tests/benchmark/` - hardware-fit, privacy-assessor, egress-warning, comparison, task-compatibility.
- `evidence/evaluation/` - the ten-case matrix drives these directly.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/evaluation/`
- `src/app/api/v1/comparisons/`
- `src/app/api/v1/readiness/`
- `src/app/api/v1/readiness/[id]/` (FOLDER.md)
- `src/app/compare/`
- `src/components/compare/`
- `src/modules/benchmark/core/`
- `src/modules/benchmark/core/entities/`

src/modules/benchmark/infrastructure/providers/ - the adapters

Work package: benchmark and provider layer.

What this folder is

Nine files: one base class, four provider adapters, a registry, the error vocabulary, and two transport helpers.

The shape is deliberate. A subclass implements exactly one thing - how to stream one generation out of one provider (`streamOnce`). Timing, timeout enforcement, error classification, iteration and narrowing to the shared result type all happen in the base class, so **the four adapters cannot drift apart in how they measure or how they fail**.

What happened here

Unchanged in this pass, which is worth noting given how much moved around them. The adapters were already correct; what changed was what the layers above them were allowed to conclude from their output.

Two properties they already had are what made the rest possible. Every adapter takes its `fetch` and its clock by **injection**, so the whole suite runs with no network and no real timers. And **nothing here throws across the module boundary**: a failed iteration is still a measurement, and the reliability score is computed from how many iterations succeeded - so swallowing a failure would inflate it.

Files

File	What it is
<code>BaseProvider.ts</code>	328 lines. Timing, timeouts, error classification, iteration. <code>ProviderFailureError</code> and its brand
<code>RecordedProvider.ts</code>	Replays measurements the visitor's browser already took against its own Ollama, so <code>BenchmarkRunner</code> scores them with the same cold-start isolation, summary, hardware fit and readiness as a server-measured run. No fallback: a cloud model is not a substitute measurement of the visitor's machine. <code>residencyProbe()</code> hands back the before/after readings the browser took
<code>OllamaProvider.ts</code>	167 lines. The local adapter. <code>POST /api/generate</code> with <code>stream: true</code> returns newline-delimited JSON, one object per token, with durations in nanoseconds . It streams even though the tokens are never displayed, because time-to-first-token cannot be measured from a buffered response - with <code>stream: false</code> the first byte and the last byte arrive together
<code>GroqProvider.ts</code>	176 lines. OpenAI-compatible SSE. Usage is omitted from streamed chunks unless <code>stream_options: { include_usage: true }</code> is sent. Groq also reports <code>x_groq.usage.completion_time</code> in seconds - server-side generation time excluding queueing, a better throughput denominator than our wall clock
<code>GeminiProvider.ts</code>	182 lines. <code>:streamGenerateContent</code> with <code>alt=sse</code> . <code>usageMetadata</code> repeats across chunks, so the last one wins. The key travels in the <code>x-goog-api-key</code> header , never the query string - a URL lands in server logs and proxy logs; a header does not
<code>DemoProvider.ts</code>	134 lines. The documented substitute the acceptance criteria allow when no real provider is reachable. It produces no measurements : every figure is labelled <code>simulated</code> , it is never registered unless <code>BENCHMARK_ALLOW_DEMO=true</code> , and the runner marks any run that touched it. Deterministic on purpose, so a demo is reproducible
<code>ProviderRegistry.ts</code>	152 lines. Which adapters exist, which are usable here, in what order to try them. Built from a <code>BenchmarkConfig</code> rather than reading <code>process.env</code> itself, so a test can construct any environment without mutating global state
<code>errors.ts</code>	199 lines. The published failure vocabulary. Every provider failure is mapped to one of these codes before it leaves an adapter. <code>docs/local-model-setup.md</code> lists them and the API returns their statuses
<code>http.ts</code>	151 lines. SSE and line-delimited stream readers, <code>tokensPerSecond</code> , the injected clock. A deliberate copy of the vision module's equivalent rather than a shared import, so the two modules can change independently

File	What it is
types.ts	129 lines. The measurement vocabulary: MeasurementProvenance, MeasurementStatus, ProviderUsage. Every number that leaves this module carries a label saying how it was obtained

Connected folders

- ../../core/ports/ - the AIProvider interface.
- ../ - config.ts builds the registry.
- src/app/api/v1/providers/ - the catalogue endpoint.
- tests/benchmark/ - one suite per adapter, plus provider-errors and provider-registry.
- docs/benchmark/ - the research log cites the official streaming contracts for all three.

src/modules/benchmark/infrastructure/repositories/ - persistence

Work package: benchmark and provider layer, over the integration layer's schema.

What this folder is

The only code in the benchmark module that talks to Prisma.

What happened here

`PrismaBenchmarkContext.ts` shrank from 46 lines to 35, because the device lookup went with the device table. `PrismaBenchmarkRepository.ts` gained the warmup flag on iteration rows and the nullable `hardwareFit`.

Three mapping rules hold here and are written into the files:

- The port speaks `camelCase`; the database is `snake_case`. The `@map()` directives in `schema.prisma` do that translation, so nothing here has to.
- `evidence` and `limitations` are `Json` columns, typed by Prisma as `JsonValue`, cast back to `string[]` on the way out. The entity declares `string[]` and the writer only ever puts strings in.
- **Nothing here swallows an error.** A caller that wants to degrade gracefully when the database is down decides that for itself - and `RunBenchmark` does exactly that, returning the measurement with `persisted: false`. A repository that silently reported success would be worse than one that fails loudly.

Files

File	What it is
<code>PrismaBenchmarkRepository.ts</code>	255 lines. The <code>BenchmarkRepository</code> implementation - saving runs, iterations and readiness scores, and reading them back
<code>PrismaBenchmarkContext.ts</code>	36 lines. The narrow lookup the use case needs: who owns the workload, and what <code>uuid</code> a provider slug maps to. Two small queries with <code>select</code> clauses rather than full row fetches - the prompt column on a workload can be large

Connected folders

- `../../core/ports/` - the interface implemented.
- `../../application/use-cases/` - the caller that decides how to degrade.
- `src/lib/` - the `PrismaClient` singleton.
- `prisma/` - the schema and its `@map()` directives.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/core/entities/`
- `src/modules/benchmark/infrastructure/`

src/app/api/v1/providers/models/ - a cloud provider's model list

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/providers/models?provider=gemini|groq - which models the key for that vendor may run. The cloud counterpart of local-runtime.

What happened here

New. The provider step used to say a cloud vendor "does not publish its catalogue over the API" and offered a free-text box with a hardcoded list of suggested names, one of which was already out of date. Both vendors do publish it, under the same key used for generation. This route asks, narrows the answer to text generators (audio, embedding and safety models are counted and left out), and marks which accept an image so the vision page can narrow further. The visitor's own key is used when they set one; otherwise the server's. The query is validated with Zod; no key appears in the response.

Files

File	What it is
route.ts	GET. Reads the provider name, builds a CloudCatalog, returns models with context_window and supports_vision

Connected folders

- src/modules/benchmark/infrastructure/ - CloudCatalog.ts and visitor-keys.ts.
- src/app/api/v1/providers/ - the parent listing.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/providers/
- src/app/api/v1/

`src/app/api/v1/readiness/[id]/`

`src/app/api/v1/readiness/[id]/` - one readiness score

Work package: benchmark and provider layer.

What this folder is

`GET /api/v1/readiness/[id]` - the readiness score recorded for one benchmark.

What happened here

Two details in this route are easy to get wrong and are pinned by comments in the file.

`[id]` is the **benchmark** id, not the readiness-score id. `readiness_scores` has a unique `benchmark_id`, and the caller holds a benchmark id.

The stored `limitations` array carries the assumption lines the run was scored under, each prefixed `ASSUMPTION:.` They are split back out here so a client does not need to know about the prefix - **a score must never be read without the caveats it was computed under.**

What the score itself means changed underneath this route: `hardwareFit` can now be `null` where it used to always be 50, and `privacyScore` is gone from the average entirely.

Files

File	What it is
<code>route.ts</code>	74 lines. GET, returning the score, its breakdown, and its assumptions separated from its limitations

Connected folders

- `src/modules/benchmark/core/services/` - `ReadinessCalculator.ts` and `HardwareFitAssessor.ts`.
- `prisma/migrations/` - the migration that made `hardware_fit` nullable.
- `src/components/dashboard/` - `RunResults.tsx`, which renders it.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/readiness/`
- `src/modules/benchmark/`

`src/app/api/v1/session-log/share/`

`src/app/api/v1/session-log/share/` - consent-gated sharing

Work package: benchmark and provider layer.

What this folder is

GET previews exactly what sharing would send. POST sends it, on an explicit confirmation.

What happened here

New, and the two design rules in it are the reason it is a separate route rather than a flag on the export.

The preview is not a description of the payload - it IS the payload. Both verbs call the same builder. Anything else drifts, and a preview that has drifted from what actually gets sent is worse than no preview.

POST requires confirm: true in the body. A request without it is refused and returns the preview instead, so "share" can never happen as a side effect of some other call. The consent wording the user agreed to is stored on the row, rather than living only in whatever the interface happened to say that day.

There is **no UI for this endpoint yet**.

Files

File	What it is
<code>route.ts</code>	170 lines. GET preview, POST send-on-confirmation

Connected folders

- `src/app/api/v1/session-log/` - the export it shares.
- `src/core/logging/` - `buildSharePayload` and `SHARE_CONSENT_STATEMENT`.
- `prisma/migrations/` - the migration that created `shared_findings`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/history/`

`src/app/api/v1/vision-benchmarks/dataset/`

`src/app/api/v1/vision-benchmarks/dataset/` - the built-in dataset, for a browser

Work package: vision benchmark.

What this folder is

`GET /api/v1/vision-benchmarks/dataset` describes the reference dataset - samples, labels, prompt and version, manifest digest - and `dataset/images/<file>` serves one fixture image.

What happened here

New with hosting. The built-in run used to be a server action calling the server's Ollama. Hosted, there is no such Ollama; the only one a visitor can measure is their own, and only their browser can reach it. So the browser needs what the server's run-service reads from disk, and these two routes hand it over. Only a file the manifest names is served, matched against the manifest's own list rather than joined onto a directory.

Files

File	What it is
<code>route.ts</code>	GET. The manifest as the executor expects it, each sample with the URL its image is fetched from
<code>images/[file]/route.ts</code>	GET. One PNG, <code>image/png</code> , cacheable for the session

Connected folders

- `src/components/vision/` - `builtInDataset.ts`, the browser-side run these serve.
- `datasets/vision-benchmark/` - the files.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/vision-benchmarks/`